

Manual Técnico Completo - CI Inovador 2026

Sistemas Digitais, Arquitetura de Computadores e Inglês Instrumental

Este documento consolida os conceitos das aulas 1 a 36 em uma única fonte técnica. O texto foi estruturado para ser pesquisável e autocontido no NotebookLM: cada conceito inclui definição, funcionamento, relações com outros tópicos, fórmulas, exemplos e limites de validade.

O conteúdo está organizado por dependência conceitual, e não pela ordem cronológica das aulas.

Mapa de cobertura

Aulas	Conteúdo consolidado
1 e 2	sistemas numéricos, complemento de 2, BCD, Gray, ASCII e detecção de erros
3 a 6	portas, álgebra booleana, níveis lógicos, formas canônicas e portas universais
7 a 9	Karnaugh, minimização e síntese com portas básicas, NAND e NOR
10	MUX, DEMUX, decoder, encoder e comparadores
11 a 16	latches, flip-flops, registradores, contadores, FSM e temporização
17 a 19	ponto fixo, IEEE 754, somadores, subtratores, shifters, flags e ULA
20 a 22	eletricidade básica, MOS, CMOS, potência, atraso e hazards elétricos
23 e 24	memórias, bancos de registradores, ROM, SRAM, DRAM, PLDs, CPLDs e FPGAs
25 a 27	algoritmos, ISA, assembly, formatos de instrução, datapath e controle
28 e 29	cache, hierarquia, prefetch e memória virtual
30 a 33	pipeline, hazards, branch prediction e FPU
34 e 35	assembly prático, flags, pilha, funções, otimização e depuração
36	inglês instrumental e leitura técnica

Notação

Símbolo	Significado
A' ou NOT A	complemento lógico de A
A.B ou AB	AND
A+B	OR
A XOR B	ou exclusivo
Q	estado atual
Q+	próximo estado
[R1]	conteúdo da memória no endereço guardado em R1
EA	endereço efetivo
n	largura em bits

1. Representação de dados e aritmética binária

1.1 Sistemas posicionais

Em uma base b , cada posição possui peso b elevado a um expoente inteiro:

$$N = \text{soma de } d_i \cdot b^i$$

No binário, $b=2$ e cada dígito d_i é 0 ou 1. Para 101101 em base 2:

$$\begin{aligned} 101101 &= 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 32 + 8 + 4 + 1 \\ &= 45 \end{aligned}$$

Os pesos depois da vírgula binária são negativos:

$$\begin{aligned} 2^{-1} &= 1/2 \\ 2^{-2} &= 1/4 \\ 2^{-3} &= 1/8 \end{aligned}$$

Logo:

$$\begin{aligned} 101,101 \text{ em base 2} \\ &= 4 + 1 + 1/2 + 1/8 \\ &= 5,625 \text{ em base 10} \end{aligned}$$

Conversão de decimal inteiro para binário

O método geral usa divisões sucessivas por 2. Os restos são lidos do último para o primeiro:

$$\begin{aligned} 45 / 2 &= 22, \text{ resto } 1 \\ 22 / 2 &= 11, \text{ resto } 0 \\ 11 / 2 &= 5, \text{ resto } 1 \\ 5 / 2 &= 2, \text{ resto } 1 \\ 2 / 2 &= 1, \text{ resto } 0 \\ 1 / 2 &= 0, \text{ resto } 1 \\ 45 \text{ decimal} &= 101101 \text{ binário} \end{aligned}$$

Também é possível decompor o valor em potências de 2. Para 45:

$$45 = 32 + 8 + 4 + 1$$

Os pesos 32, 8, 4 e 1 recebem bit 1.

Conversão de fração decimal para binário

Multiplica-se repetidamente a parte fracionária por 2. A parte inteira obtida em cada passo fornece o próximo bit:

$$\begin{aligned} 0,625 \cdot 2 &= 1,250 \rightarrow \text{bit } 1, \text{ resta } 0,250 \\ 0,250 \cdot 2 &= 0,500 \rightarrow \text{bit } 0, \text{ resta } 0,500 \\ 0,500 \cdot 2 &= 1,000 \rightarrow \text{bit } 1, \text{ resta } 0 \\ 0,625 \text{ decimal} &= 0,101 \text{ binário} \end{aligned}$$

Nem toda fração decimal termina em binário. O valor 0,1 decimal, por exemplo, produz expansão binária infinita. Isso explica parte dos erros de arredondamento em ponto flutuante.

1.2 Binário e hexadecimal

Um dígito hexadecimal representa quatro bits:

Hex	Binário	Hex	Binário
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Exemplo:

1010 1111 0011 1010 binário = AF3A hexadecimal

Hexadecimal é uma notação compacta. Ele não altera o padrão de bits nem a interpretação signed ou unsigned.

1.3 Largura, faixa e interpretação

Com n bits existem 2^n padrões distintos. O significado desses padrões depende da representação.

Sem sinal

faixa unsigned = 0 até $2^n - 1$

Em quatro bits, a faixa é 0 a 15.

Sinal e magnitude

O bit mais significativo representa o sinal e os demais, a magnitude. Essa representação possui +0 e -0 e complica a aritmética. Ela é útil como contraste conceitual, mas não é a representação inteira mais comum em processadores.

Complemento de 1

O negativo é obtido invertendo todos os bits. Também possui dois zeros:

+0 = 0000
-0 = 1111

Complemento de 2

O negativo é obtido invertendo os bits e somando 1:

$C2(X) = 2^n - X$, para X diferente de zero

Exemplo de -5 em quatro bits:

+5 = 0101
inverter = 1010
somar 1 = 1011
-5 em C2 = 1011

O peso do bit mais significativo é negativo:

valor = $-b[n-1] \cdot 2^{(n-1)}$ + soma de $b[i] \cdot 2^i$, para $i=0$ até $n-2$

Assim:

1101 em C2 de 4 bits = $-8 + 4 + 1 = -3$

A faixa é assimétrica:

complemento de 2 = $-2^{(n-1)}$ até $2^{(n-1)}-1$

Em quatro bits, a faixa é -8 a +7. O menor negativo não possui positivo representável na mesma largura:

1000 = -8

Negar 1000 em quatro bits produz novamente 1000, pois +8 não cabe.

Extensão e truncamento

Para ampliar um unsigned, acrescentam-se zeros à esquerda. Para ampliar um signed em complemento de 2, replica-se o bit de sinal:

1011 em 4 bits, valor -5
11111011 em 8 bits, valor -5

Truncar bits superiores equivale a tomar o resultado módulo 2^n . O valor pode mudar e informações de overflow podem ser perdidas.

1.4 Soma, subtração, carry, borrow e overflow

Regras elementares:

0+0 = 0
0+1 = 1
1+1 = 10
1+1+1 = 11

Em largura fixa:

resultado armazenado = resultado matemático mod 2^n

O carry de saída pertence à interpretação unsigned. Exemplo em quatro bits:

1111 + 0001 = 1 0000

O resultado armazenado é 0000 e Cout=1. Como unsigned, 15+1 excedeu 15.

Overflow signed

Carry final e overflow signed não são a mesma condição. Em complemento de 2, ocorre overflow na soma quando operandos de mesmo sinal produzem resultado de sinal oposto:

0111 + 0011 = 1010
+7 + +3 aparenta produzir -6

Outra forma:

V = carry que entra no bit de sinal XOR carry que sai do bit de sinal

Operandos de sinais diferentes não causam overflow signed em uma soma, embora possam produzir carry.

Subtração por complemento de 2

$A-B = A + C2(B) = A + B' + 1$

Exemplo de 3-5 em quatro bits:

```

3      = 0011
5      = 0101
C2(5) = 1011
soma   = 1110
1110   = -2

```

Na convenção ARM:

```

C=1 após A-B -> não houve borrow
C=0 após A-B -> houve borrow

```

Outras arquiteturas podem nomear ou polarizar essa indicação de maneira diferente.

Na subtração signed, ocorre overflow quando A e B possuem sinais diferentes e o resultado possui sinal diferente de A.

1.5 BCD 8421

BCD codifica cada dígito decimal separadamente com quatro bits:

Decimal	BCD	Decimal	BCD
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

Os padrões 1010 a 1111 são inválidos como um único dígito BCD.

```

59 decimal = 0101 1001 BCD
12 decimal = 0001 0010 BCD
12 decimal = 1100 em binário puro

```

Na soma de um dígito BCD, adiciona-se 0110 se o resultado exceder 9 ou se houver carry para o dígito seguinte:

```

7 + 5
0111 + 0101 = 1100, que é inválido em BCD
1100 + 0110 = 1 0010

```

O carry forma a dezena 1 e 0010 representa a unidade 2.

1.6 Código Gray

No código Gray, valores consecutivos diferem em apenas um bit. Isso reduz ambiguidades quando vários bits físicos não mudam exatamente ao mesmo tempo, como em encoders rotativos e na travessia de contadores entre domínios de clock.

Binário para Gray:

```

G[MSB] = B[MSB]
G[i] = B[i+1] XOR B[i]
G = B XOR (B >> 1)

```

Exemplo:

```

B = 1011
B>>1 = 0101
G = 1110

```

Gray para binário usa XOR acumulado:

```
B[MSB] = G[MSB]
B[i] = B[i+1] XOR G[i]
```

Para G=1110:

```
B3=1
B2=1 XOR 1=0
B1=0 XOR 1=1
B0=1 XOR 0=1
B=1011
```

Gray não é uma base posicional. Não se aplicam pesos 8,4,2,1 diretamente ao código Gray.

1.7 ASCII, paridade, checksum e CRC

ASCII padrão possui sete bits. Armazenamento ou transmissão pode usar um oitavo bit para extensão ou paridade.

Âncoras úteis:

```
'0' = 48 decimal = 0x30
'A' = 65 decimal = 0x41
'a' = 97 decimal = 0x61
diferença entre minúscula e maiúscula = 32
```

Caractere e valor numérico são entidades distintas:

```
valor inteiro 7
caractere '7' = 0x37 = 55 decimal
```

Paridade

Na paridade par, o bit adicional faz o total de bits 1 ser par. Na paridade ímpar, faz o total ser ímpar. Um único bit de paridade detecta qualquer quantidade ímpar de inversões, mas pode deixar passar uma quantidade par. Ele normalmente detecta, mas não corrige.

Checksum

Checksum é uma família de técnicas, não um único algoritmo. Em uma variante de oito bits com soma total zero:

```
S = soma dos bytes mod 256
checksum = (-S) mod 256
```

Se S=0, o checksum é 0. Checksum simples pode falhar quando erros se compensam ou quando a ordem dos bytes não participa do cálculo.

Exemplo:

```
dados 3,5,2 -> soma 10
dados 4,4,2 -> soma 10
```

Uma soma simples não distingue essas sequências.

CRC

CRC trata a mensagem como polinômio binário e calcula um resto por divisão polinomial. Ele oferece cobertura de erros muito superior à paridade ou a uma soma simples, mas também é um mecanismo de detecção, não garantia absoluta de correção.

2. Sinais lógicos, portas e álgebra booleana

2.1 Do nível elétrico ao valor lógico

Os valores H e L indicam faixas reconhecidas como alto e baixo. Eles não representam tensões universais exatas; os limiares dependem da tecnologia, tensão de alimentação e condições elétricas.

Além de 0 e 1, aparecem estados especiais:

Símbolo	Significado
Z	alta impedância; o circuito não força a linha
X em simulação	valor desconhecido ou conflito
X em especificação/Karnaugh	don't care, quando explicitamente declarado

Z não é zero. Uma linha em Z pode ser dirigida por outro componente ou assumir valor por pull-up ou pull-down.

Buffer tri-state ativo alto:

EN=1 -> Y=D
EN=0 -> Y=Z

Se dois drivers impõem simultaneamente valores opostos no mesmo barramento, existe contenção. Isso pode gerar corrente elevada e valor lógico indefinido. Um X desconhecido em simulação não deve ser convertido automaticamente em don't care.

2.2 Portas lógicas

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

NOT: $Y=A'$
XOR: $Y=A'B+AB'$
XNOR: $Y=AB+A'B'$

OR significa pelo menos uma entrada 1. XOR significa uma quantidade ímpar de entradas 1 para sua extensão usual; com duas entradas, significa entradas diferentes.

A precedência convencional é:

NOT antes de AND antes de OR

Parênteses removem ambiguidades. A barra ou apóstrofo precisa ter alcance bem definido:

$A'+B$ é diferente de $(A+B)'$

Uma tabela com n variáveis possui 2^n linhas. Para converter expressão em circuito, implementam-se primeiro as operações internas. Para converter circuito em expressão, nomeiam-se as saídas intermediárias.

Exemplo:

$N1 = B'$
 $N2 = A.N1$
 $Y = N2+C$

Logo:

$Y = AB'+C$

2.3 Leis da álgebra booleana

Lei	OR	AND
identidade	$A+0=A$	$A.1=A$
dominação	$A+1=1$	$A.0=0$
idempotência	$A+A=A$	$A.A=A$
complemento	$A+A'=1$	$A.A'=0$
comutativa	$A+B=B+A$	$AB=BA$
associativa	$A+(B+C)=(A+B)+C$	$A(BC)=(AB)C$
absorção	$A+AB=A$	$A(A+B)=A$

Dupla negação:

$$(A')' = A$$

Distributivas:

$$\begin{aligned} A(B+C) &= AB+AC \\ A+BC &= (A+B)(A+C) \end{aligned}$$

Padrões derivados:

$$\begin{aligned} AB+AB' &= A(B+B') = A \\ (A+B)(A+B') &= A+BB' = A \\ A+A'B &= (A+A')(A+B) = A+B \end{aligned}$$

Álgebra booleana não é aritmética comum:

$$1+1=1 \text{ no OR booleano}$$

Duas expressões são equivalentes quando produzem a mesma saída para todas as entradas. Isso pode ser demonstrado por transformações algébricas ou por tabela-verdade.

Simplificar uma função pode reduzir quantidade de portas, entradas de portas e níveis lógicos. Em uma implementação física, isso pode reduzir área, atraso e energia, embora detalhes de tecnologia e fanout também influenciem.

2.4 De Morgan e dualidade

$$\begin{aligned} (A+B)' &= A'B' \\ (AB)' &= A'+B' \end{aligned}$$

Para múltiplas entradas:

$$\begin{aligned} (A+B+C)' &= A'B'C' \\ (ABC)' &= A'+B'+C' \end{aligned}$$

Ao mover uma negação através de uma porta, troca-se AND por OR ou OR por AND e negam-se os termos. Esse processo é frequentemente descrito como empurrar bolhas.

Exemplo aninhado:

$$\begin{aligned} (A+B'C)' & \\ &= A' \cdot (B'C)' \\ &= A' \cdot (B+C) \end{aligned}$$

Dualidade troca:

+ por .
 . por +
 0 por 1
 1 por 0

Variáveis e complementos permanecem. O dual de $A+BC$ é $A(B+C)$. O dual de uma identidade válida também é uma identidade válida, mas uma expressão não precisa ser igual ao seu dual. Dualidade não é complemento.

2.5 NAND e NOR como portas universais

Somente NAND:

$A' = A \text{ NAND } A$
 $AB = (A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B)$
 $A+B = (A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B)$

Somente NOR:

$A' = A \text{ NOR } A$
 $A+B = (A \text{ NOR } B) \text{ NOR } (A \text{ NOR } B)$
 $AB = (A \text{ NOR } A) \text{ NOR } (B \text{ NOR } B)$

Uma SOP possui implementação natural em dois níveis NAND-NAND:

$F=P+Q$
 $F=(P' \cdot Q')'$

Uma POS possui implementação natural em dois níveis NOR-NOR:

$F=P \cdot Q$
 $F=(P'+Q')'$

NAND e NOR, diferentemente de AND e OR, não podem ser reagrupadas livremente como operações associativas. Se a biblioteca possui somente portas de duas entradas, uma função de muitas entradas precisa ser decomposta e o número de níveis pode aumentar.

Exemplo:

$F=A'B+CD$

Somente NAND:

$N0=A \text{ NAND } A$
 $N1=N0 \text{ NAND } B$
 $N2=C \text{ NAND } D$
 $F=N1 \text{ NAND } N2$

3. Formas canônicas, Karnaugh e síntese lógica

3.1 Literais, mintermos e maxtermos

Literal é uma variável direta ou negada. Termo produto é um AND de literais. Termo soma é um OR de literais.

Um mintermo de n variáveis contém todas as variáveis e vale 1 em exatamente uma combinação. Um maxtermo contém todas e vale 0 em exatamente uma combinação.

Bit da combinação	No mintermo	No maxtermo
0	variável negada	variável direta
1	variável direta	variável negada

Para ABC=101:

$m_5 = AB'C$
 $M_5 = A'+B+C'$

O índice 5 vem da interpretação binária de 101.

Forma SOP canônica:

F = soma dos mintermos em que F=1
 $F = \Sigma m(\text{índices dos 1})$

Forma POS canônica:

F = produto dos maxtermos em que F=0
 $F = \Pi M(\text{índices dos 0})$

Exemplo:

$F = \Sigma m(1, 2, 4, 6, 7)$
 $F = \Pi M(0, 3, 5)$

Todo índice possui um mintermo e um maxtermo. A função seleciona mintermos correspondentes aos 1 ou maxtermos correspondentes aos 0. Forma canônica não significa forma mínima.

Linhas não especificadas podem ser declaradas como don't care:

$d(10, 11, 12, 13, 14, 15)$

Nessas linhas, a implementação pode produzir 0 ou 1 sem violar a especificação.

3.2 Fundamento do mapa de Karnaugh

Karnaugh reorganiza a tabela-verdade em ordem Gray para que células adjacentes difiram em uma única variável. Agrupar elimina a variável que muda:

$A'B+AB = B(A'+A) = B$

Mapa de duas variáveis:

A\B	0	1
0	m0	m1
1	m2	m3

Mapa de três variáveis:

A\BC	00	01	11	10
0	m0	m1	m3	m2
1	m4	m5	m7	m6

Mapa de quatro variáveis:

AB\CD	00	01	11	10
00	m0	m1	m3	m2
01	m4	m5	m7	m6
11	m12	m13	m15	m14
10	m8	m9	m11	m10

Primeira e última linhas são adjacentes; primeira e última colunas também. Os quatro cantos formam um grupo. Diagonais e formas em L não são grupos.

3.3 Regras de agrupamento

Para SOP, agrupam-se 1. Para POS, agrupam-se 0.

tamanhos válidos = 1,2,4,8,16...

- o grupo precisa ser retangular;
- todos os valores obrigatórios devem ser cobertos;
- um grupo SOP não pode conter 0 especificado;
- um grupo POS não pode conter 1 especificado;
- grupos podem atravessar bordas;
- sobreposição é permitida;
- grupos maiores eliminam mais variáveis;
- pode existir mais de uma cobertura mínima equivalente.

Um implicante primo é um grupo que não pode ser ampliado. Um implicante primo essencial cobre ao menos um valor obrigatório que nenhum outro implicante cobre.

Número de literais restantes:

literais = $n - \log_2(\text{tamanho do grupo})$

Em quatro variáveis:

Grupo	Literais restantes
1	4
2	3
4	2
8	1
16	0

Extração de SOP

variável constante em 1 -> literal direto
 variável constante em 0 -> literal negado
 variável que muda -> desaparece

Extração de POS

variável constante em 0 -> literal direto
 variável constante em 1 -> literal negado
 variável que muda -> desaparece

Exemplos de quatro variáveis:

$\Sigma m(0,2,8,10) = B'D'$
 $\Sigma m(0,1,8,9) = B'C'$
 $\Sigma m(0,1,2,3,8,9,10,11) = B'$

No primeiro conjunto, A e C mudam, enquanto B=0 e D=0.

3.4 Don't care

Um X declarado como don't care pode:

- completar um grupo maior;
- ser ignorado;
- assumir 0 ou 1 na implementação final.

Um grupo formado somente por X não cobre nenhum valor obrigatório e não acrescenta utilidade. Em SOP, X nunca permite incluir um 0 especificado. Em POS, não permite incluir um 1 especificado.

Exemplo:

$F = \Sigma m(1,3,5,7,9,11) + d(13,15)$

Os seis 1 e os dois X formam uma oitava em que D=1:

$F = D$

Entradas BCD inválidas de 10 a 15 podem ser don't care somente quando a especificação garante que essas combinações não ocorrerão.

3.5 Cadeia completa de síntese

especificação
 -> tabela-verdade
 -> SOP ou POS canônica
 -> minimização algébrica ou Karnaugh
 -> expressão mínima
 -> implementação
 -> verificação

Análise percorre o sentido inverso:

circuito -> expressão -> tabela/comportamento

Exemplo:

$F = \Sigma m(1,3,6,7) = \Pi M(0,2,4,5)$

Uma SOP mínima é:

$F = A'C + AB$

Portas básicas:

$N0=A'$
 $N1=N0.C$
 $N2=A.B$
 $F=N1+N2$

NAND-NAND:

$N0=A \text{ NAND } A$
 $N1=N0 \text{ NAND } C$
 $N2=A \text{ NAND } B$
 $F=N1 \text{ NAND } N2$

Uma POS equivalente é:

$F=(A+C)(A'+B)$

NOR-NOR:

$N0=A \text{ NOR } A$
 $N1=A \text{ NOR } C$
 $N2=N0 \text{ NOR } B$
 $F=N1 \text{ NOR } N2$

A verificação precisa confirmar todos os 1 obrigatórios, todos os 0 obrigatórios e ignorar a saída nas linhas declaradas X.

4. Blocos combinacionais

4.1 Conceito de circuito combinacional

Em um circuito combinacional, as saídas são função apenas das entradas atuais. Não há estado interno intencional. Atrasos físicos podem fazer a saída transitar antes de estabilizar, mas não constituem memória lógica.

Enable determina se um bloco está ativo. Seleção determina qual caminho ou linha é escolhido. Sufixo _n, barra ou bolha indicam sinal ativo em nível baixo.

4.2 Multiplexador

MUX seleciona uma entre várias entradas de dados.

MUX 2:1:

$$Y = S' I_0 + S I_1$$

MUX 4:1:

$$Y = S_1' S_0' I_0 + S_1' S_0 I_1 + S_1 S_0' I_2 + S_1 S_0 I_3$$

Com n seletores, podem ser selecionadas até 2^n entradas.

MUX implementa a decomposição de Shannon:

$$F = S' \cdot F|_{S=0} + S \cdot F|_{S=1}$$

Uma função de três variáveis pode ser implementada com MUX 4:1 usando duas variáveis como seleção. Para:

$$F(A, B, C) = \Sigma m(1, 2, 6, 7)$$

Escolhendo AB como seleção:

AB	F quando C=0	F quando C=1	Entrada
00	0	1	I0=C
01	1	0	I1=C'
10	0	0	I2=0
11	1	1	I3=1

Logo:

$$I_0=C, I_1=C', I_2=0, I_3=1$$

Com MUX 8:1, A, B e C seriam seletores e cada entrada seria ligada diretamente a 0 ou 1 conforme a tabela.

4.3 DEMUX e decoder

DEMUX encaminha uma entrada de dados para uma saída selecionada. Para um DEMUX 1:4 ativo alto:

$$\begin{aligned} Y_0 &= D \cdot S_1' \cdot S_0' \\ Y_1 &= D \cdot S_1' \cdot S_0 \\ Y_2 &= D \cdot S_1 \cdot S_0' \\ Y_3 &= D \cdot S_1 \cdot S_0 \end{aligned}$$

Se D=1, ele se comporta como um decoder ativo alto.

Decoder converte um código de n bits em uma entre 2^n linhas. Para decoder 2:4:

```

D0=A'B'
D1=A'B
D2=AB'
D3=AB

```

Cada saída de um decoder completo corresponde a um mintermo. Por isso:

```
F=Sigma m(1,2,6,7)
```

pode ser implementada com decoder 3:8 e uma OR:

```
F=D1+D2+D6+D7
```

Decoder recebe um código. DEMUX recebe seleção e também um dado a encaminhar.

4.4 Encoder e encoder prioritário

Encoder faz a operação conceitual inversa do decoder: uma linha ativa gera um código.

Encoder 4:2 one-hot:

```

Y1=D2+D3
Y0=D1+D3

```

Essas equações supõem uma única entrada ativa. Se D3 e D1 estiverem ativas simultaneamente, um encoder simples gera resultado ambíguo.

Encoder prioritário escolhe a entrada de maior prioridade. Para prioridade $D3 > D2 > D1 > D0$, D3 domina as demais. Um sinal valid distingue:

```

D0 ativo -> código 00, valid=1
nenhuma entrada -> código 00, valid=0

```

Encoder decimal para BCD possui dez entradas, uma por dígito, e quatro saídas.

4.5 Comparadores

Comparador de um bit:

```

EQ=A XNOR B
GT=AB'
LT=A'B

```

Em palavras maiores, o bit mais significativo que difere decide a relação unsigned. A igualdade total é o AND das igualdades de todos os bits.

Comparação signed exige interpretar o bit de sinal. Os padrões:

```

A=1111
B=0001

```

representam 15 e 1 como unsigned, mas -1 e 1 em complemento de 2. Portanto:

```

unsigned: A>B
signed: A<B

```

Comparadores podem ser encadeados por sinais de igualdade, maior e menor vindos de blocos mais significativos.

4.6 Relações entre os blocos

Necessidade	Bloco
escolher uma fonte	MUX
encaminhar um dado a um destino	DEMUX
ativar uma linha a partir de um código	decoder
produzir código a partir de uma linha	encoder
resolver múltiplas linhas ativas	encoder prioritário
determinar igualdade ou magnitude	comparador

MUX e decoder são também estruturas universais para funções combinacionais. A diferença é o ponto de vista: MUX seleciona cofatores da função; decoder materializa mintermos.

5. Armazenamento, registradores e máquinas de estados

5.1 Realimentação, estabilidade e latch SR

Um circuito sequencial contém estado: a saída atual depende das entradas e de informação armazenada sobre o passado. O mecanismo elementar de armazenamento é a realimentação, na qual parte da saída retorna à entrada da lógica.

Em um estado estável, mantendo as entradas constantes, a realimentação confirma o valor armazenado. Q representa o estado atual e Q' sua saída complementar em operação válida.

Latch SR com NOR, entradas ativas em nível alto:

S	R	Q+	Operação
0	0	Q	mantém
1	0	1	set
0	1	0	reset
1	1	inválido	Q e Q' deixam de ser complementares

Na condição $S=R=1$, as duas saídas são forçadas a 0. Se S e R retornam quase simultaneamente a 0, pequenos atrasos podem decidir qual estado será alcançado. A condição é inválida não apenas por uma convenção de tabela, mas porque o resultado após sua liberação não é controlado.

Latch SR com NAND, entradas ativas em nível baixo:

S _n	R _n	Q+	Operação
1	1	Q	mantém
0	1	1	set
1	0	0	reset
0	0	inválido	Q e Q' deixam de ser complementares

Sufixo _n, barra sobre o nome ou bolha no símbolo indicam ativo baixo.

Exemplo de rastreamento de SR-NOR, iniciando em Q=0:

```
entradas: 00, 10, 00, 01, 00
Q:        0,  1,  1,  0,  0
```

Nos intervalos 00, a saída depende do estado anterior.

5.2 Latch D

O latch D elimina o comando contraditório do SR. Para um SR-NOR:

```
S=EN.D
R=EN.D'
```

Como D e D' não podem ser 1 simultaneamente, $S=R=1$ não ocorre.

Latch D com enable ativo alto:

```
EN=1 -> Q+=D
EN=0 -> Q+=Q
```

Enquanto $EN=1$, o latch é transparente: mudanças em D podem aparecer em Q durante todo o intervalo ativo. Ao desativar EN, o último valor é mantido.

Para enable ativo baixo:

```
ENn=0 -> transparente
ENn=1 -> mantém
```

Latch é sensível a nível. Isso o distingue do flip-flop disparado por borda.

5.3 Flip-flops e clock

Em sistemas síncronos, o clock concentra as mudanças de estado em instantes definidos.

posedge ou seta para cima -> borda 0 para 1
negedge ou bolha no clock -> borda 1 para 0

Flip-flop D:

na borda ativa: $Q^+ = D$
entre bordas: Q mantém

Flip-flop D com enable ativo alto:

$$Q^+ = EN \cdot D + EN' \cdot Q$$

Se $EN=0$ na borda, o estado permanece. Se $EN=1$, D é capturado.

Flip-flop T:

T	Q^+
0	Q
1	Q'

$$Q^+ = Q \text{ XOR } T$$

Com $T=1$ permanente, Q alterna a cada borda e divide a frequência por dois.

Flip-flop JK:

J	K	Q^+
0	0	Q
0	1	0
1	0	1
1	1	Q'

$$Q^+ = J \cdot Q' + K' \cdot Q$$

Reset, preset e prioridade

reset -> força $Q=0$
preset ou set -> força $Q=1$

Controle síncrono espera a borda ativa. Controle assíncrono age sem esperar clock. A prioridade entre reset, preset, enable e entrada normal faz parte da especificação do elemento. Reset assíncrono é útil para inicialização, mas sua liberação precisa respeitar cuidados de temporização.

Metastabilidade

Se D muda dentro da janela de setup/hold ao redor da borda, o flip-flop pode demorar um tempo imprevisível para resolver em 0 ou 1. Metastabilidade não é um terceiro estado lógico estável. Um sincronizador com dois ou mais flip-flops reduz a probabilidade de a condição se propagar, mas não a torna matematicamente zero.

5.4 Registradores e deslocamento

Um registrador de n bits é um conjunto de n flip-flops com clock comum. Carga paralela atualiza todos os bits na mesma borda:

antes: Q=0011, D=1010
 após borda com enable ativo: Q=1010

Com enable de palavra, todos os bits carregam ou todos mantêm.

Tipos de registrador de deslocamento:

Tipo	Entrada	Saída
SISO	serial	serial
SIPO	serial	paralela
PISO	paralela	serial
PIPO	paralela	paralela

A ordem observada depende de qual estágio é a saída e da direção do deslocamento. Se a saída serial é Q0, o primeiro bit observado de uma carga paralela é o valor inicial de Q0.

Ring counter

Um registrador ring realimenta diretamente uma extremidade na outra.

n flip-flops -> n estados no ciclo one-hot válido

Exemplo para quatro bits:

0001 -> 0010 -> 0100 -> 1000 -> 0001

Ele precisa ser inicializado em um estado válido. Em 0000 pode permanecer preso.

Johnson counter

O Johnson realimenta o complemento da extremidade:

n flip-flops -> 2n estados no ciclo válido padrão

Exemplo conceitual de quatro bits:

0000 -> 1000 -> 1100 -> 1110 -> 1111
 -> 0111 -> 0011 -> 0001 -> 0000

Ring e Johnson são registradores sequenciais específicos; não seguem automaticamente a sequência de um contador binário.

5.5 Máquinas de estados finitos

Estado é um resumo da parte do passado necessária para determinar o comportamento futuro.

Q e X -> Q+
 Q e, às vezes, X -> Z

Uma FSM contém:

lógica de próximo estado
 registrador de estado
 lógica de saída

Antes da borda, a lógica combinacional calcula Q+. Na borda, o registrador copia Q+ para Q.

Moore e Mealy

Moore:

$Z=g(Q)$

A saída é associada ao estado.

Mealy:

$$Z = g(Q, X)$$

A saída é associada à transição e pode reagir combinacionalmente à entrada dentro do ciclo.

Moore frequentemente usa mais estados e reduz a dependência direta entre entrada e saída. Isso não garante ausência absoluta de glitch, porque a decodificação dos bits de estado ainda é lógica física.

Codificação de estados

Codificação binária mínima:

$$\text{bits} = \text{ceil}(\log_2 N)$$

One-hot:

$$N \text{ estados} \rightarrow N \text{ flip-flops}$$

One-hot usa mais armazenamento, mas pode simplificar lógica, especialmente em FPGA. A codificação não é única; mudar códigos muda as equações, não o comportamento abstrato.

Estados binários não usados podem:

- ser tratados como don't care para simplificação;
- ser redirecionados ao estado inicial para autorrecuperação.

As duas escolhas possuem compromissos diferentes entre lógica mínima e robustez.

Síntese com diferentes flip-flops

Com flip-flop D:

$$D = Q^+$$

Com flip-flop T:

$$T = Q \text{ XOR } Q^+$$

Excitação JK:

Q	Q+	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Detector de sequência 10 em Mealy

Estados:

S0, Q=0: nenhum 1 relevante
S1, Q=1: último bit foi 1

Transições:

Q	X	Q+	Z
0	0	0	0
0	1	1	0
1	0	0	1
1	1	1	0

Equações:

$$D=X$$

$$Z=Q.X'$$

A detecção ocorre na transição que recebe 0 após um 1.

Detector de sequência 10 em Moore

S0=00: nenhum 1 relevante
 S1=01: último bit foi 1
 S2=10: sequência detectada, Z=1
 11: não usado

Para os estados válidos:

$$D1=Q1'.Q0.X'$$

$$D0=X$$

$$Z=Q1.Q0'$$

Moore precisa de um estado adicional para representar a saída. Reset define S0 como estado inicial. Sem estado inicial, uma sequência de entradas não determina unicamente o rastreo.

Detecção sobreposta preserva um sufixo que também é prefixo da próxima ocorrência. Por exemplo, ao detectar 101 dentro de 10101, o último 1 pode iniciar a próxima detecção.

6. Contadores e temporização sequencial

6.1 Contadores binários

Contador percorre uma sequência de estados a cada pulso de clock. Módulo é a quantidade de estados antes da repetição.

n flip-flops $\rightarrow$ até 2^n estados
mínimo para módulo M em binário $\rightarrow \lceil \log_2 M \rceil$

Contador crescente de três bits:

000 $\rightarrow$ 001 $\rightarrow$ 010 $\rightarrow$ 011
 $\rightarrow$ 100 $\rightarrow$ 101 $\rightarrow$ 110 $\rightarrow$ 111 $\rightarrow$ 000

Contador decrescente percorre a ordem inversa.

No contador binário completo:

$fQ0 = f_{clock}/2$
 $fQ1 = f_{clock}/4$
 $fQi = f_{clock}/2^{(i+1)}$

Exemplo para clock de 8 kHz:

$Q0 = 4 \text{ kHz}$
 $Q1 = 2 \text{ kHz}$
 $Q2 = 1 \text{ kHz}$

Essa relação não deve ser aplicada automaticamente a contadores truncados, ring ou Johnson.

6.2 Ripple e síncrono

No contador assíncrono ou ripple, apenas o primeiro flip-flop recebe o clock principal; cada estágio seguinte é acionado pela saída anterior. Os bits não mudam simultaneamente. Durante uma transição podem aparecer estados intermediários que não pertencem à sequência lógica estável.

Para n estágios com atraso aproximado tpd , a última saída pode exigir tempo da ordem de:

$n \cdot tpd$

Quatro estágios com 10 ns por estágio podem exigir aproximadamente 40 ns para estabilização completa.

No contador síncrono, todos os flip-flops recebem o mesmo clock. A lógica combinacional determina quais alternam.

Com FF T, contador crescente:

$T0 = 1$
 $T1 = Q0$
 $T2 = Q1 \cdot Q0$
 $Ti = \text{produto de todos os bits inferiores}$

Contador decrescente:

$T0 = 1$
 $T1 = Q0'$
 $T2 = Q1' \cdot Q0'$

Com FF D, contador crescente de dois bits:

$D0 = Q0'$
 $D1 = Q1 \text{ XOR } Q0$

Um contador é uma FSM cuja transição depende apenas do estado atual, salvo sinais de enable, direção, carga ou reset.

6.3 Contadores módulo M

Contador BCD ou década possui módulo 10:

0000 até 1001

Os padrões 1010 a 1111 não pertencem ao ciclo válido. Um projeto robusto pode redirecioná-los para 0000.

Módulo menor pode ser obtido detectando o primeiro estado fora da sequência e resetando. Para módulo 6:

estados válidos: 000 a 101
primeiro estado inválido: 110
detecção exata de 110: Q2.Q1.Q0'

Reset assíncrono pode interromper a contagem imediatamente após a detecção e gerar pulsos estreitos. Reset síncrono espera a próxima borda e possui comportamento temporal mais controlado.

6.4 Caminho registrador a registrador

O modelo fundamental é:

FF de origem -> lógica combinacional -> FF de destino

Tempos:

tCQ = clock-to-Q do registrador de origem
tcomb = atraso da lógica combinacional
tsetup = estabilidade exigida antes da borda de captura
thold = estabilidade exigida depois da borda de captura

Período e frequência:

$T=1/f$
 $T(ns)=1000/f(MHz)$

6.5 Restrição de setup e caminho crítico

Sem skew:

$T_{clock} \geq t_{CQmax} + t_{combmax} + t_{setup}$
 $F_{max} = 1/T_{clock_min}$

O caminho com maior atraso entre registradores é o caminho crítico de setup.

Exemplo:

tCQmax=2 ns
tcombmax=8 ns
tsetup=1 ns
Tmínimo=11 ns
Fmax=1000/11 MHz = 90,9 MHz

Slack de setup é a diferença entre tempo disponível e tempo necessário:

$slack_setup = T_{clock} - (t_{CQmax} + t_{combmax} + t_{setup})$

Em 50 MHz, T=20 ns. Se o caminho exige 19 ns:

slack_setup=1 ns

Slack negativo indica violação.

6.6 Restrição de hold e caminho mínimo

Sem skew:

```
tCQmin+tcombmin >= thold
```

Hold usa atrasos mínimos, pois o risco é o novo dado chegar cedo demais.

Exemplo:

```
tCQmin=0,8 ns
tcombmin=0,5 ns
thold=2 ns
chegada mínima=1,3 ns
slack_hold=1,3-2=-0,7 ns
```

Existe violação de 0,7 ns.

Diminuir a frequência aumenta a distância até a próxima borda e ajuda setup, mas normalmente não altera o intervalo ao redor da mesma borda responsável por hold.

Correções típicas:

```
setup -> lógica mais rápida, menos níveis, período maior, pipeline
hold -> inserir atraso, ajustar roteamento ou distribuição de clock
```

6.7 Clock skew

Definindo:

```
skew = chegada do clock no destino - chegada na origem
```

As restrições tornam-se:

```
setup:
Tclock >= tCQmax+tcombmax+tsetup-skew

hold:
tCQmin+tcombmin >= thold+skew
```

Interpretação:

```
skew positivo -> ajuda setup e piora hold
skew negativo -> piora setup e ajuda hold
```

Essa conclusão depende da convenção de sinal declarada.

6.8 Pipeline, temporização e metastabilidade

Pipeline divide uma lógica longa em estágios menores. Isso pode reduzir tcomb por estágio e permitir clock mais rápido, mas acrescenta:

- registradores;
- clock-to-Q e setup em cada estágio;
- energia de clock;
- latência em ciclos;
- complexidade de controle.

O estágio mais lento continua limitando o período.

Entradas assíncronas, botões, sinais externos e cruzamentos entre clocks podem mudar perto da borda. Sincronizadores reduzem a probabilidade de metastabilidade atingir a lógica seguinte. Dados multibit exigem protocolos adequados; sincronizar cada bit independentemente pode produzir uma palavra incoerente.

7. Representações numéricas e datapath aritmético

7.1 Ponto fixo

Com B bits totais e F bits fracionários:

```
escala=2^F
valor_real=inteiro_bruto/2^F
inteiro_bruto_aproximado=valor_real.2^F
resolução=2^-F
```

O ponto binário não é armazenado; sua posição é uma convenção.

Faixas:

```
unsigned:
0 até (2^B-1)/2^F

signed em C2:
-2^(B-1)/2^F até (2^(B-1)-1)/2^F
```

Exemplo signed com oito bits e F=4:

```
bits=11110000
inteiro C2=-16
valor=-16/16=-1
```

Soma na mesma escala:

```
1,5 -> 24
2,25 -> 36
24+36=60
60/16=3,75
```

Multiplicação de dois valores com F bits fracionários produz escala $2^{(2F)}$:

```
1,5 -> 24
2,0 -> 32
produto_bruto=768
768/256=3,0
```

Para retornar a F=4:

```
768 >> 4 = 48
48/16=3,0
```

O deslocamento descarta bits e pode exigir arredondamento. Mais F aumenta resolução, mas reduz a faixa inteira se B permanecer fixo. A convenção Qm.n precisa declarar se o bit de sinal está incluído em m.

7.2 Ponto flutuante e IEEE 754

Ponto flutuante representa:

```
sinal . significando . 2^expoente
```

Números binários normais possuem forma:

```
1.fração . 2^e
```

O 1 inicial é implícito e não ocupa bit no campo de fração.

IEEE 754 simples:

```
1 bit de sinal
8 bits de expoente
23 bits de fração
bias=127
```

IEEE 754 dupla:

```
1 bit de sinal
11 bits de expoente
52 bits de fração
bias=1023
```

Para normal:

```
valor=(-1)^s . 1.fração . 2^(E-bias)
```

Exemplo de 13,25:

```
13,25=1101,01 binário
      =1,10101 . 2^3

s=0
E=3+127=130=10000010
fração=10101000...
```

Casos especiais em precisão simples:

Expoente	Fração	Classe
0	0	zero com sinal
0	diferente de 0	subnormal, significando 0.f
1 a 254	qualquer	normal, significando 1.f
255	0	infinito
255	diferente de 0	NaN

Subnormal permite underflow gradual, com precisão reduzida próximo de zero. O espaçamento entre floats aumenta com o expoente; ponto fixo possui espaçamento uniforme.

7.3 Arredondamento binário

Bits auxiliares:

```
G = guard, primeiro bit descartado
R = round, segundo bit descartado
S = sticky, OR de todos os bits posteriores
```

No modo mais próximo com empate para par, incrementa-se o significando mantido quando:

```
G=1 e (R=1 ou S=1 ou LSB mantido=1)
```

O último termo resolve o empate exatamente no meio, escolhendo resultado cujo último bit mantido é par.

Modos IEEE comuns:

```
mais próximo, empate para par
em direção a zero
em direção a +infinito
em direção a -infinito
```

Inexact indica que bits relevantes foram descartados. O bit sticky do arredondamento não é a flag sticky de status da FPU.

7.4 Half-adder, full-adder e ripple-carry

Half-adder:

A	B	S	Cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$S = A \oplus B$
 $Cout = A.B$

Full-adder:

$S = A \oplus B \oplus Cin$
 $Cout = A.B + A.Cin + B.Cin$

Construção com dois half-adders:

$X = A \oplus B$
 $C1 = A.B$
 $S = X \oplus Cin$
 $C2 = X.Cin$
 $Cout = C1 + C2$

No ripple-carry, Cout de cada estágio alimenta Cin do seguinte. Sob um modelo uniforme:

atraso da soma do MSB = $(n-1)t_{carry} + t_{soma}$
 atraso do Cout final = $n.t_{carry}$

Cout é uma saída separada da palavra de n bits.

7.5 Propagate, generate e carry look-ahead

$P_i = A_i \oplus B_i$
 $G_i = A_i.B_i$
 $S_i = P_i \oplus C_i$
 $C_{(i+1)} = G_i + P_i.C_i$

Expansão:

$C1 = G0 + P0.C0$
 $C2 = G1 + P1.G0 + P1.P0.C0$

Carry look-ahead calcula carries por lógica paralela, reduzindo a cadeia serial. O ganho de velocidade custa área, fan-in e roteamento.

7.6 Subtração e somador-subtrator

Half-subtractor:

$D = A \oplus B$
 $Bout = A'.B$

Full-subtractor:

```
D=A XOR B XOR Bin
Bout=A'.B+A'.Bin+B.Bin
```

Uma estrutura de soma pode realizar subtração:

```
Bentrada=B XOR SUB
C0=SUB

SUB=0 -> A+B
SUB=1 -> A+B'+1=A-B
```

Exemplo:

```
0011-0101=1110
```

Interpretação:

```
unsigned: houve borrow
signed: 3-5=-2, sem overflow
```

7.7 Comparadores

Igualdade de uma palavra:

```
EQ=produto de XNOR bit a bit
```

Para dois bits:

```
GT=A1.B1' + (A1 XNOR B1).A0.B0'
LT=A1'.B1 + (A1 XNOR B1).A0'.B0
```

O primeiro bit mais significativo diferente decide a magnitude unsigned.

Subtração também fornece comparação:

```
Z=1 -> igualdade
unsigned ARM-like: C=0 -> A<B
signed: N XOR V=1 -> A<B
```

O mesmo padrão pode ter ordens diferentes:

```
1010 versus 0111
unsigned: 10>7
signed C2: -6<7
```

7.8 Shifters

```
LSL -> desloca à esquerda, zeros entram à direita
LSR -> desloca à direita, zeros entram à esquerda
ASR -> desloca à direita, replica o bit de sinal
ROL/ROR -> bits descartados retornam pelo outro lado
```

Exemplo:

```
1011 ASR 1 = 1101
1011 LSR 1 = 0101
```

LSL por k equivale a multiplicar por 2^k somente se os bits relevantes não forem perdidos. LSR corresponde à divisão unsigned. ASR preserva o sinal, mas a divisão de negativos pode arredondar de modo diferente de uma linguagem que trunca em direção a zero.

Barrel shifter usa níveis de multiplexadores para escolher deslocamentos de 1,2,4 e outras potências, realizando deslocamento variável em uma única passagem combinacional.

7.9 ULA e flags

ULA é um bloco combinacional:

operandos + controle -> resultado + flags

Uma implementação pode calcular operações em paralelo e usar um MUX de saída.

Exemplo de controle:

Controle	Operação
00	AND
01	OR
10	ADD
11	SUB

Flags:

Z -> resultado igual a zero
 N -> bit mais significativo
 C -> carry; na subtração ARM-like, sem borrow
 V -> overflow signed

N sozinho não determina comparação signed após subtração; a condição correta de menor usa N XOR V. C e V descrevem interpretações diferentes do mesmo padrão de bits.

8. Eletricidade, MOS, CMOS, potência e atraso

8.1 Grandezas elétricas

Tensão é diferença de potencial entre dois pontos. Corrente é fluxo de carga elétrica. Resistência relaciona tensão e corrente:

$$\begin{aligned} V &= R \cdot I \\ I &= V/R \\ R &= V/I \end{aligned}$$

Em um caminho aberto não existe condução ideal. Em um caminho fechado pode existir corrente, desde que haja diferença de potencial e uma trajetória condutora.

Potência é taxa de transferência de energia:

$$\begin{aligned} P &= V \cdot I \\ P &= E/t \\ E &= P \cdot t \end{aligned}$$

Capacitância relaciona carga e tensão:

$$Q = C \cdot V$$

Unidades:

Grandeza	Unidade
tensão	volt, V
corrente	ampere, A
resistência	ohm
potência	watt, W
energia	joule, J
capacitância	farad, F

Prefixos:

$$\begin{aligned} m &= 10^{-3} \\ \text{micro} &= 10^{-6} \\ n &= 10^{-9} \\ p &= 10^{-12} \\ f &= 10^{-15} \end{aligned}$$

Exemplos:

$$\begin{aligned} V &= 5 \text{ V}, R = 1 \text{ kohm} \\ I &= 5/1000 = 5 \text{ mA} \\ \\ V &= 3,3 \text{ V}, I = 10 \text{ mA} \\ P &= 3,3 \cdot 0,010 = 33 \text{ mW} \\ \\ P &= 50 \text{ mW durante } 20 \text{ s} \\ E &= 0,050 \cdot 20 = 1 \text{ J} \end{aligned}$$

Valores lógicos são associados a faixas de tensão, não a valores físicos exatos universais. VDD é a alimentação positiva e GND é a referência.

8.2 MOS como chave

Na abstração digital:

NMOS conduz com $gate=1$
 PMOS conduz com $gate=0$

NMOS é usado naturalmente na rede pull-down, criando caminho para GND. PMOS é usado na pull-up, criando caminho para VDD.

Inversor CMOS:

A	PMOS	NMOS	Y
0	ligado	desligado	1
1	desligado	ligado	0

Em estado estável ideal não existe caminho permanente de VDD a GND. Em circuitos reais há corrente de fuga. Durante uma transição, PMOS e NMOS podem conduzir brevemente.

A entrada MOS apresenta capacitância. Mudar uma saída exige carregar ou descarregar capacitâncias das portas seguintes, dos fios e dos parasitas do próprio circuito.

8.3 Redes CMOS complementares

Uma porta CMOS estática possui:

PUN = rede pull-up de PMOS
 PDN = rede pull-down de NMOS

Na PDN:

NMOS em série -> AND das condições de condução
 NMOS em paralelo -> OR das condições de condução

Se a PDN conduz quando $G=1$:

$Y=G'$

Na PUN, série e paralelo são duais. O procedimento geral para analisar uma rede é:

1. determinar a condição G em que existe caminho pela PDN;
2. escrever $Y=G'$;
3. aplicar De Morgan;
4. confirmar que a PUN é a rede dual.

Exemplo:

PDN: A em série com B paralelo C
 $G=A(B+C)$
 $Y=[A(B+C)]'$
 $Y=A'+B'C'$

A PUN possui o ramo controlado por A em paralelo com uma série controlada por B e C.

NAND e NOR

NAND de n entradas:

PDN: n NMOS em série
 PUN: n PMOS em paralelo

NOR de n entradas:

PDN: n NMOS em paralelo
 PUN: n PMOS em série

Contagem ideal:

```
inversor -> 2 transistores
NAND ou NOR de n entradas -> 2n transistores
AND -> NAND mais inversor
OR -> NOR mais inversor
```

Se nenhuma rede conduz, a saída fica em alta impedância. Se ambas conduzem permanentemente, existe caminho indesejado VDD-GND. Condução breve durante uma transição é um fenômeno dinâmico diferente.

8.4 Potência estática e dinâmica

Componentes principais:

```
potência dinâmica de carga/descarga
corrente de curto-circuito durante transição
potência estática por leakage
```

Potência estática:

```
Pest aproximada=VDD.Ileak
```

Potência dinâmica:

```
Pdin aproximada=alpha.C.VDD^2.f
```

Onde:

- alpha representa atividade de comutação segundo a convenção do modelo;
- C é a capacitância efetiva;
- VDD é a tensão de alimentação;
- f é a frequência de eventos ou clock relevante.

Relações:

```
C dobra -> Pdin dobra
f dobra -> Pdin dobra
alpha dobra -> Pdin dobra
VDD dobra -> Pdin quadruplica
```

Energia armazenada em capacitor:

```
Ecap=1/2.C.V^2
```

No carregamento resistivo ideal de 0 a V, a fonte fornece:

```
Efonte=C.V^2
```

Metade fica armazenada no capacitor e metade é dissipada durante o carregamento. Na descarga, a energia armazenada também é dissipada. A expressão $E_{\text{fonte}}=C.V^2$ refere-se especificamente a esse evento de carregamento.

Atalhos de unidade:

```
1 pF.V^2 = 1 pJ
1 fF.V^2 = 1 fJ
```

8.5 Atraso de propagação, fanout e caminho crítico

Atraso de propagação é o intervalo entre a mudança da entrada e a estabilização da saída.

```
tPLH -> saída muda de 0 para 1
tPHL -> saída muda de 1 para 0
```

Esses valores podem ser diferentes porque redes NMOS e PMOS possuem características distintas.

Fanout é a quantidade de entradas dirigidas por uma saída. Mais fanout geralmente significa maior capacitância, maior atraso e maior potência dinâmica.

A carga também inclui:

- capacitância de fios;
- parasitas de difusão;
- entradas de portas seguintes;
- buffers e interconexões.

Em uma cadeia, os atrasos se acumulam ao longo do caminho. Se três caminhos têm 5 ns, 9 ns e 6 ns:

```
caminho crítico=9 ns
```

Em lógica síncrona:

```
Tclock >= tCQmax+tcombmax+tsetup
```

O caminho mínimo, e não o crítico máximo, determina risco de hold.

Layout influencia comprimento de fios, capacitância, resistência, fanout e congestionamento. Otimizar a expressão lógica não garante, sozinho, o melhor atraso físico.

8.6 Glitches e hazards elétricos

Glitch é um pulso transitório observado. Hazard é a estrutura lógica e temporal que permite esse pulso.

Tipos:

```
hazard estático-1 -> deveria permanecer 1, cai brevemente
hazard estático-0 -> deveria permanecer 0, sobe brevemente
hazard dinâmico -> deveria mudar uma vez, muda várias
```

A causa típica é a reconvergência de caminhos com atrasos diferentes. Uma variável se divide, percorre caminhos distintos e volta a influenciar a mesma saída.

Exemplo:

```
F=AB+A'C
```

Se B=C=1, o valor lógico ideal é 1 tanto antes quanto depois de A mudar. Entretanto, um caminho pode desligar antes do outro ligar. O termo de consenso BC cobre a transição:

```
F=AB+A'C+BC
```

O termo é redundante na álgebra funcional, mas útil fisicamente.

Forma de consenso para POS:

```
(A+B)(A'+C)
-> adicionar (B+C)
```

No Karnaugh:

```
estático-1 -> 1 adjacentes devem compartilhar cobertura
estático-0 -> 0 adjacentes devem compartilhar cobertura
```

Em um caminho de dados síncrono, um glitch que desaparece antes da janela de captura pode não alterar o estado. Em clock, reset assíncrono, set assíncrono ou enable sensível a nível, um pulso curto pode ter efeito imediato.

Hazard elétrico é diferente de hazard de pipeline. O primeiro decorre de atrasos físicos em lógica; o segundo decorre de recursos, dependências ou controle entre instruções.

9. Memórias e lógica programável

9.1 Organização de uma memória

Uma memória possui:

```
endereço -> seleciona uma posição
data-in -> dado a escrever
data-out -> dado lido
controles -> enable, leitura, escrita e, em alguns casos, clock
```

Uma palavra é a largura acessada por posição.

Com n bits de endereço e W bits por palavra:

```
posições=2^n
capacidade em bits=2^n.W
capacidade em bytes=2^n.W/8
```

Exemplo:

```
8 bits de endereço
palavra de 16 bits
posições=256
capacidade=4096 bits=512 bytes
```

Outro:

```
12 bits de endereço
palavra de 8 bits
posições=4096
capacidade=4096 bytes
```

KB pode significar 1000 bytes em convenção decimal. KiB significa 1024 bytes. Cálculos binários didáticos frequentemente usam 1024, mas a convenção precisa ser identificada.

Leitura e escrita

Leitura seleciona endereço e habilita a saída. Escrita seleciona endereço, fornece data-in e ativa write-enable. Chip-enable permite desativar o componente e compartilhar barramentos.

Uma escrita síncrona ocorre em borda de clock. Uma leitura pode ser combinacional ou síncrona, conforme o componente.

9.2 Registrador e banco de registradores

Um registrador guarda uma palavra. Banco de registradores contém várias palavras pequenas e rápidas.

```
N registradores -> ceil(log2 N) bits de índice
```

Oito registradores exigem três bits de seleção.

Um datapath frequentemente usa duas portas de leitura e uma de escrita. Conceitualmente:

```
decoder -> seleciona qual registrador recebe escrita
MUXes -> selecionam registradores para as portas de leitura
```

RegWrite controla a atualização. Registrador de destino zero ou uma porta com valor não utilizado pode possuir comportamento especial conforme a ISA.

9.3 ROM

ROM é não volátil. Ela pode implementar tabela-verdade:

entradas da função -> endereço
saídas da função -> palavra armazenada

Para três entradas existem oito endereços. Cada endereço guarda $F(A,B,C)$. Várias saídas podem ser armazenadas na mesma palavra.

Família:

PROM -> normalmente programada uma vez
EPROM -> apagada por luz ultravioleta
EEPROM -> apagada/regravada eletricamente
Flash -> apagada/regravada em blocos

9.4 SRAM e DRAM

Propriedade	SRAM	DRAM
célula	estrutura biestável	carga em capacitor
refresh	não	sim
velocidade	maior	menor
densidade	menor	maior
custo por bit	maior	menor
volatilidade	volátil	volátil
uso típico	cache	memória principal

SRAM é estática porque mantém o bit enquanto alimentada sem refresh periódico. Isso não significa não volátil.

DRAM perde carga gradualmente e precisa de refresh. O controlador organiza linhas, bancos e temporização de acesso. Random access significa que posições podem ser selecionadas diretamente por endereço; não significa latência idêntica em todas as situações.

Hierarquia típica:

registradores
-> cache em SRAM
-> memória principal em DRAM
-> armazenamento não volátil

Quanto mais perto do processador, menor capacidade, menor latência e maior custo por bit.

9.5 PROM, PLA e PAL

A implementação clássica em SOP possui:

plano AND -> termos produto
plano OR -> soma dos produtos

Dispositivo	Plano AND	Plano OR
PROM	fixo, decoder completo	programável
PLA	programável	programável
PAL	programável	fixo/limitado

PROM gera todos os 2^n mintermos, o que cresce exponencialmente com n .

PLA gera apenas os produtos necessários e pode compartilhá-los entre saídas. Exemplo:

$F1 = AB + CD$
 $F2 = AB + A' C$

O produto AB pode alimentar as duas saídas.

PAL tende a ser mais simples e rápida, mas limita quantidade e conexão dos produtos disponíveis por saída.

Tecnologias de configuração incluem fusível, antifuse, EEPROM, Flash e SRAM.

9.6 CPLD

CPLD reúne vários blocos semelhantes a PAL ou macrocélulas conectados por matriz programável. É comum em:

- decodificação;
- glue logic;
- interfaces;
- FSM de controle.

Em comparação com FPGA, costuma possuir menor capacidade e menos recursos especializados, mas pode oferecer atraso mais previsível. Isso é uma tendência de arquitetura, não garantia universal.

9.7 FPGA

FPGA contém:

LUTs
flip-flops
multiplexadores
carry chains
rede de interconexão
blocos de I/O
RAM, DSP, PLL e outros recursos, conforme o dispositivo

LUT de k entradas possui 2^k posições e implementa qualquer função de até k variáveis.

LUT de duas entradas para AND:

AB	conteúdo
00	0
01	0
10	0
11	1

Ela pode ser entendida como uma pequena memória seguida de seleção. Funções com mais entradas que a LUT são decompostas em várias LUTs.

O flip-flop próximo da LUT implementa registradores, FSM e pipeline. Carry chains aceleram somadores e contadores. Interconexão e roteamento frequentemente dominam o atraso físico.

Fluxo de implementação

HDL ou esquemático
-> síntese
-> mapeamento em recursos
-> placement
-> routing
-> análise de temporização
-> bitstream
-> configuração

Placement escolhe posições físicas. Routing escolhe conexões. HDL descreve hardware concorrente; não é simplesmente um programa sequencial executado linha por linha.

Tecnologia de configuração

SRAM-based:

volátil, reconfigurável, precisa carregar bitstream ao ligar

Flash-based:

não volátil, reprogramável conforme tecnologia

antifuse:

não volátil, normalmente programável uma vez

Reconfiguração parcial depende de suporte específico do dispositivo e do fluxo. FPGA reconfigurável não significa que toda implementação muda dinamicamente em execução.

9.8 FPGA, CPLD e ASIC

Aspecto	CPLD	FPGA	ASIC
flexibilidade	reconfigurável	muito reconfigurável	função fixa
capacidade típica	menor	alta	definida no projeto
custo inicial	baixo/moderado	moderado	muito alto
custo unitário em grande volume	maior	maior	menor
desempenho/energia potencial	moderado	bom	melhor potencial
tempo de desenvolvimento	curto	curto/moderado	longo

ASIC possui alto custo não recorrente de projeto e fabricação, mas pode obter melhor área, desempenho e energia em grande volume. FPGA oferece prototipagem e atualização sem fabricar novo chip.

10. Arquitetura, ISA e assembly

10.1 Camadas de abstração

```

algoritmo
-> linguagem de alto nível
-> assembly
-> código de máquina
-> sinais de controle
-> hardware

```

Arquitetura ou ISA é o contrato visível ao software:

- instruções;
- registradores;
- tipos de dados;
- modos de endereçamento;
- comportamento de exceções e operações.

Organização ou microarquitetura é a implementação do contrato:

- datapath;
- unidade de controle;
- pipeline;
- cache;
- preditores;
- unidades funcionais.

Duas CPUs podem executar a mesma ISA com microarquiteturas diferentes.

Algoritmo procedural combina:

```

sequência
seleção, como if/else
iteração, como while/for
chamada de procedimento ou função

```

Assembly expressa operações elementares desse algoritmo. A sintaxe, os nomes e a ordem dos operandos dependem da ISA.

10.2 Operação e operandos

```

instrução = opcode + operandos

```

Tipos principais de operando:

Tipo	Local do valor
registrador	dentro da CPU
imediato	codificado na instrução
memória	posição selecionada por endereço

Notação genérica:

```

R2    = conteúdo do registrador R2
[R2]  = conteúdo da memória no endereço guardado em R2
#5    = constante imediata 5

```

Modelo load/store:

```

LOAD/LDR -> memória para registrador
STORE/STR -> registrador para memória
ULA      -> opera principalmente em registradores

```

Exemplo:

```

LDR R1,[R2]    ; R1=memória[R2]
ADD R1,R1,#3    ; R1=R1+3
STR R1,[R2]    ; memória[R2]=R1

```

Modificar R1 não altera automaticamente a memória. O STORE é necessário.

Classes:

```

aritmética -> ADD, SUB, MUL
lógica     -> AND, OR, XOR, NOT
deslocamento -> LSL, LSR, ASR
transferência -> LOAD, STORE, MOVE
comparação -> CMP, TEST
fluxo     -> branch, call, return

```

Labels são nomes simbólicos de endereços. O montador calcula a codificação do alvo ou deslocamento.

10.3 Formatos de instrução

Campos possíveis:

```

opcode | rd | rs1 | rs2
opcode | rd | rs1 | imediato
opcode | registrador de dado | base | deslocamento

```

Dimensionamento:

```

N registradores -> ceil(log2 N) bits por campo
M operações     -> ceil(log2 M) bits para opcode mínimo
imediato unsigned k bits -> 0 a 2^k-1
imediato signed k bits  -> -2^(k-1) a 2^(k-1)-1

```

Em instrução de largura fixa existe compromisso: mais bits para registradores, opcode ou função deixam menos bits para imediato e deslocamento.

Modelos de operandos:

Modelo	Exemplo conceitual
três endereços	$R1 = R2 + R3$
dois endereços	$R1 = R1 + R2$
um endereço	acumulador = acumulador + mem[A]
zero endereços	máquina de pilha usa topo implicitamente

R, I e J são nomes comuns para famílias de formatos, mas a codificação exata e até o formato separado de branch dependem da ISA.

10.4 Modos de endereçamento

Modo	Significado
imediato	valor está na instrução
registrador	valor está em registrador
direto/absoluto	instrução contém endereço
indireto por registrador	registrador contém endereço
base+offset	EA=base+deslocamento
indexado	EA=base+índice
índice escalado	EA=base+índice.tamanho
PC-relative	alvo=PC de referência+deslocamento
implícito	operando definido pela própria operação
pilha	operandos no topo indicado por SP

Exemplo:

```
R2=1000
LOAD R1, [R2, #12]
EA=1012
```

Para vetor de words de 32 bits:

```
tamanho=4 bytes
EA=base+índice.4
índice<<2 = índice.4
```

Offset é normalmente medido em bytes, a menos que a ISA declare escala diferente.

Extensão

Sign extension replica o bit de sinal de um imediato signed. Zero extension acrescenta zeros e é usada para valores unsigned ou operações lógicas conforme a ISA.

```
8 bits: 11111100 = -4 signed
sign-extend para 16: 1111111111111100
zero-extend para 16: 0000000011111100 = 252
```

10.5 Endereço PC-relative e deslocamento escalado

PC-relative torna código mais fácil de realocar, pois o alvo é expresso em relação à instrução.

Se um deslocamento signed de 12 bits representa words de quatro bytes:

```
faixa em words=-2048 a +2047
faixa em bytes=-8192 a +8188
```

O extremo positivo é um passo menor em magnitude porque complemento de 2 é assimétrico.

O valor exato do PC usado, a escala e o tamanho da instrução dependem da ISA.

10.6 Byte addressing, largura, alinhamento e endianness

Em memória byte-addressable, cada endereço identifica um byte.

Larguras ARM-like:

```
LDR/STR -> word
LDRB/STRB -> byte
LDRH/STRH -> halfword
```

Uma leitura unsigned estreita normalmente recebe zero extension. Uma variante signed recebe sign extension.

Alinhamento natural coloca:

```
halfword em múltiplo de 2
word de 4 bytes em múltiplo de 4
doubleword de 8 bytes em múltiplo de 8
```

Acesso desalinhado pode ser permitido, mais lento, dividido em operações ou gerar exceção, conforme a arquitetura.

Para a palavra 0x12345678 iniciando no endereço A:

Endereço	Little-endian	Big-endian
A	0x78	0x12
A+1	0x56	0x34
A+2	0x34	0x56
A+3	0x12	0x78

Endianness muda a ordem dos bytes na memória, não a ordem dos bits dentro de cada byte.

10.7 Atualização da base

Sintaxe ARM-like:

```
[R2,#4]    -> usa R2+4, não altera R2
[R2,#4]!   -> pre-index, usa R2+4 e grava R2=R2+4
[R2],#4    -> post-index, usa R2 antigo e depois grava R2=R2+4
```

Shift embutido em endereço não precisa alterar o registrador índice:

```
LDR R0,[R4,R1,LSL #2]
```

calcula $R4+(R1 \ll 2)$, mas pode preservar R1.

Exemplo integrado:

```
R4=0x1000
R1=3
EA=R4+(R1<<2)=0x100C
LDR R2,[R4,R1,LSL #2]
ADD R2,R2,#1
STR R2,[R4,#16]      ; escreve em 0x1010
```

Memória precisa ser atualizada conceitualmente no instante de cada STORE, pois acessos posteriores podem observar o novo valor.

10.8 CMP, flags e branches

```
CMP A,B -> calcula A-B somente para atualizar flags
```

O resultado não é armazenado em A.

Flags:

```
Z -> resultado zero
N -> bit mais significativo do resultado
C -> carry; em subtração ARM-like, sem borrow
V -> overflow signed
```

Branches:

Relação	Instrução	Condição
igual	BEQ	Z=1
diferente	BNE	Z=0
signed menor	BLT	N diferente de V
signed maior/igual	BGE	N igual a V
signed maior	BGT	Z=0 e N=V
signed menor/igual	BLE	Z=1 ou N diferente de V
unsigned menor	BLO/BCC	C=0
unsigned maior/igual	BHS/BCS	C=1
unsigned maior	BHI	C=1 e Z=0
unsigned menor/igual	BLS	C=0 ou Z=1

BNE não compara sozinho; ele consulta flags produzidas por CMP, SUBS ou outra instrução anterior.

O mesmo padrão pode produzir relações distintas:

```
A=1111
B=0001

unsigned: 15>1
signed: -1<1
```

Comparações signed usam N e V; unsigned usa C e Z.

10.9 Laços

Exemplo:

```
loop:
    SUBS R1,R1,#1
    BNE loop
```

SUBS atualiza flags. BNE repete enquanto o resultado não for zero.

Tradução conceitual de while:

```
while (R1 != 0) {
    corpo;
    R1=R1-1;
}
```

É necessário considerar o valor inicial. Se o teste ocorre somente depois do decremento, iniciar em zero pode causar underflow e número inesperado de iterações.

10.10 Funções, LR e pilha

```
BL alvo -> desvia e salva retorno em LR
BX LR   -> retorna
SP      -> ponteiro do topo da pilha
PUSH    -> reserva espaço e salva
POP     -> restaura e libera espaço
```

Função folha não chama outra função e pode manter LR sem salvá-lo, conforme a convenção. Função não folha executa outro BL; sem salvar LR, perde seu próprio endereço de retorno.

Exemplo:

```
func:
    PUSH {R4, LR}
    BL outra
    POP {R4, LR}
    BX LR
```

Em pilha descendente com words de quatro bytes:

```
SP inicial=0x1000
PUSH {R4, LR} salva 8 bytes
SP novo=0x0FF8
```

Caller-saved deve ser preservado pelo chamador se ainda for necessário. Callee-saved precisa ser restaurado pela função chamada.

Convenção ARM-like comum:

```
R0-R3 -> argumentos/temporários, caller-saved
R0 -> retorno
R4-R11 -> callee-saved
SP -> pilha
LR -> retorno
PC -> próxima instrução
```

Detalhes variam conforme a ABI.

10.11 Otimização e depuração

Tempo de CPU:

```
tempo aproximado=contagem de instruções.CPI.Tclock
```

Contagem estática é a quantidade de instruções no arquivo. Contagem dinâmica é a quantidade realmente executada.

Uma otimização precisa definir a métrica:

- tempo;
- ciclos;
- tamanho de código;
- energia;
- tráfego de memória.

Transformações possíveis:

- manter temporários em registradores;
- remover loads/stores redundantes quando semanticamente seguro;
- retirar cálculo invariável de laço;
- atualizar ponteiro em vez de recalculando endereço;
- desenrolar laço;
- reorganizar instruções independentes;
- melhorar localidade.

Desenrolamento aumenta código e pressão por registradores. Quantidades que não são múltiplas do fator exigem laço de resto. Falta de registradores pode provocar spill para a pilha.

Leituras e escritas em memória mapeada em E/S, compartilhada ou com efeitos observáveis não podem ser removidas ou reordenadas livremente.

Invariante útil para soma de vetor:

R2 = soma dos elementos já processados
 R0 = endereço do próximo elemento
 R1 = quantidade restante

Depuração procura o primeiro estado divergente:

PC
 registradores
 flags
 endereço efetivo
 memória
 SP
 LR

Sintomas:

Sintoma	Possíveis causas
uma volta extra	posição do teste ou contador
função não retorna	LR ou SP corrompido
valor muda após chamada	registrador caller-saved não preservado
branch contradiz os números	flags antigas ou signed/unsigned incorreto
acesso incorreto	EA, escala, alinhamento ou largura

Um LDR não significa automaticamente cache miss. Ele pode produzir hit, miss, stall de memória ou hazard load-use, dependendo de camadas diferentes.

11. Datapath e unidade de controle

11.1 Componentes

PC
 memória de instruções
 banco de registradores
 ULA
 memória de dados
 MUXes
 somadores do PC
 extensor de sinal ou zero
 registradores intermediários ou de pipeline

Datapath transporta, guarda e transforma dados. Controle gera sinais que comandam o datapath.

11.2 Ciclo lógico de uma instrução

fetch -> decode -> execute -> memory -> write-back

Nem toda instrução usa funcionalmente todas as etapas. STORE não escreve em registrador; branch não precisa produzir write-back normal.

Sinais típicos:

RegWrite
 MemRead
 MemWrite
 ALUSrc
 MemToReg
 Branch
 Jump
 PCSrc
 ALUOp
 ALUControl

Tabela genérica RISC:

Instrução	RegWrite	ALUSrc	ULA	MemRead	MemWrite	MemToReg	Branch
ADD/SUB	1	0	ADD/SUB	0	0	0	0
ADDI	1	1	ADD	0	0	0	0
LOAD	1	1	ADD endereço	1	0	1	0
STORE	0	1	ADD endereço	0	1	X	0
BEQ	0	0	SUB/comparação	0	0	X	1

X significa que o sinal não afeta o resultado daquela instrução. Em hardware real, controles são definidos para valores seguros.

11.3 Caminhos das instruções

Tipo R

PC -> memória de instruções
 registradores fonte -> ULA
 resultado da ULA -> registrador destino

ADDI

```
registrador fonte
+ imediato estendido
-> ULA
-> registrador destino
```

LOAD

```
base + offset -> ULA calcula EA
memória[EA] -> dado
dado -> registrador destino
```

LOAD usa ULA, pois o endereço precisa ser calculado.

STORE

```
base + offset -> ULA calcula EA
registrador de dado -> memória[EA]
RegWrite=0
```

BEQ

```
registradores -> subtração/comparação
Zero e Branch -> escolha de PC
```

A subtração serve à decisão; não é escrita como resultado normal.

JUMP

```
alvo calculado ou codificado -> MUX do próximo PC
```

O caminho sequencial frequentemente soma tamanho da instrução ao PC. Em ISA de instruções fixas de 32 bits, isso pode ser PC+4.

11.4 Controle principal e controle da ULA

Controle principal decodifica opcode e produz sinais gerais. Controle da ULA combina ALUOp com campo funct ou outras informações para escolher ADD, SUB, AND e demais operações.

Hardwired:

```
lógica fixa
menor latência potencial
menor flexibilidade para mudanças complexas
```

Microprogramado:

```
memória de controle
sequência de microinstruções
maior flexibilidade
```

Essas são estratégias de implementação da unidade de controle, não conjuntos diferentes de instruções por definição.

11.5 Monociclo, multiciclo e pipeline

Monociclo:

```
cada instrução completa em um ciclo
período limitado pelo caminho da instrução mais lenta
```

Se LOAD atravessa busca, leitura de registrador, ULA, memória e write-back, todas as instruções pagam um clock suficiente para esse pior caminho.

Multiciclo:

instrução usa várias etapas em ciclos diferentes
recursos podem ser reutilizados
controle frequentemente é uma FSM

Pipeline:

etapas de instruções diferentes se sobrepõem
registradores isolam estágios

Monociclo, multiciclo e pipeline podem implementar a mesma ISA. Eles diferem em temporização, uso de recursos, CPI e complexidade.

12. Hierarquia, cache, memória virtual e prefetch

12.1 Hierarquia e localidade

```
registradores
-> cache em SRAM
-> memória principal em DRAM
-> armazenamento
```

Cache mantém cópias de blocos de níveis mais lentos.

```
hit -> bloco encontrado
miss -> bloco ausente
hit rate=hits/acessos
miss rate=misses/acessos=1-hit rate
```

Localidade:

```
temporal -> dado usado tende a ser reutilizado
espacial -> endereços próximos tendem a ser usados
```

Linha ou bloco é a unidade transferida entre níveis.

12.2 Mapeamento

Direto

Cada bloco possui uma única linha possível:

```
linha=número do bloco mod número de linhas
```

É simples, mas blocos que mapeiam à mesma linha podem alternar e causar misses de conflito.

Associativo por conjunto

```
conjunto=número do bloco mod número de conjuntos
```

O bloco pode ocupar qualquer via do conjunto. Cache 4-way possui quatro linhas por conjunto.

Totalmente associativo

O bloco pode ocupar qualquer linha. Reduz conflitos, mas exige comparar mais tags.

12.3 Tag, índice, offset e valid

```
offset=log2(bytes por bloco)
conjuntos=capacidade/(bloco.vias)
índice=log2(conjuntos)
tag=bits do endereço-índice-offset
```

Fluxo:

```
índice seleciona conjunto
-> valid confirma que a entrada contém dado válido
-> tags são comparadas
-> offset escolhe byte ou palavra dentro do bloco
```

Tag igual com valid=0 não é hit.

Exemplo:

```

endereço de 12 bits
cache direta com 8 linhas
bloco de 4 bytes

offset=log2 4=2 bits
índice=log2 8=3 bits
tag=12-3-2=7 bits

```

Em cache direta de quatro linhas, blocos 0 e 4 usam a mesma linha:

```

0 mod 4=0
4 mod 4=0

```

Acesso alternado 0,4,0,4 pode gerar conflito contínuo mesmo sem exceder a capacidade total.

12.4 Tipos de miss, substituição e escrita

Misses clássicos:

```

compulsório -> primeiro acesso ao bloco
capacidade -> conjunto de trabalho excede a cache
conflito -> mapeamento impede coexistência

```

Políticas de substituição:

```

LRU
pseudo-LRU
FIFO
aleatória

```

LRU exato mantém ordem de uso e pode custar muita lógica em alta associatividade. Pseudo-LRU aproxima a decisão.

Escrita:

```

write-through -> atualiza nível seguinte em cada escrita
write-back -> altera apenas cache e marca dirty

```

Ao expulsar linha dirty em write-back, o bloco precisa ser escrito no nível seguinte. Write buffer pode desacoplar parte da latência.

Política de alocação também importa:

```

write-allocate -> miss de escrita traz bloco para cache
no-write-allocate -> escrita segue ao nível inferior sem alocar

```

Combinações comuns são write-back com write-allocate e write-through com no-write-allocate, mas não são obrigatórias.

12.5 Tempo médio de acesso

```

AMAT=hit time+miss rate.miss penalty

```

Exemplo:

```

hit time=1 ns
miss rate=5%=0,05
miss penalty=50 ns

AMAT=1+0,05.50=3,5 ns

```

Hit rate não é velocidade da cache. Uma cache pode ter hit rate maior e hit time também maior.

12.6 Memória virtual

Cada processo utiliza um espaço de endereços virtuais. A MMU traduz páginas virtuais para frames físicos.

```
endereço virtual=VPN+page offset
endereço físico=PFN+o mesmo page offset
```

Se a página possui 2^p bytes, o offset possui p bits. A tradução troca VPN por PFN e preserva o deslocamento dentro da página.

Tabela de páginas é associada ao processo. Uma PTE pode conter:

- número do frame;
- bit presente/válido;
- permissões de leitura, escrita e execução;
- bits de uso e modificação;
- outros estados da arquitetura.

Dois processos podem usar o mesmo endereço virtual e chegar a frames físicos diferentes.

12.7 TLB e page fault

TLB guarda traduções recentes:

```
VA -> consulta TLB
TLB hit -> PFN disponível
TLB miss -> page-table walk
```

Após o walk:

```
PTE presente -> atualizar TLB e continuar
PTE não presente ou inválida -> page fault
```

TLB miss não é page fault. O primeiro é ausência da tradução na pequena cache de traduções. O segundo transfere controle ao sistema operacional porque a página exige tratamento.

Page fault pode:

- alocar página sob demanda;
- carregar conteúdo do armazenamento;
- mapear arquivo;
- indicar acesso inválido ou violação de proteção.

Memória virtual existe mesmo sem swap. Suas funções incluem:

```
abstração e relocação
isolamento e proteção
compartilhamento controlado
mapeamento de arquivos
```

Container usa mecanismos de isolamento de processos e ambiente. Memória virtual é especificamente o mecanismo de tradução, proteção e organização do espaço de endereços.

Cache guarda dados e instruções. TLB guarda traduções. Implementações reais podem consultar tradução e cache em paralelo em certos arranjos, mas os papéis permanecem distintos.

12.8 Prefetch

Busca sob demanda começa quando o dado é solicitado. Prefetch antecipa a transferência para esconder latência.

Tipos:

```
next-line -> bloco B sugere B+1
stream -> acompanha sequência detectada
stride -> próximo endereço=atual+passo
```

Pode atuar sobre instruções ou dados e ser iniciado por hardware ou por instrução/software.

Métricas:

```
accuracy=prefetches úteis/prefetches realizados
coverage=misses evitados/misses sem prefetch
timeliness=adequação do momento de chegada
```

Distância é quão à frente buscar. Grau é quantos blocos buscar por evento.

Prefetch conservador tende a preservar accuracy e cobrir menos misses. Prefetch agressivo pode aumentar coverage, mas consumir banda e poluir cache.

Exemplos:

Vetor sequencial

Linha de 16 bytes e elementos int de 4 bytes:

```
4 elementos por linha
```

Next-line ou stream pode antecipar a linha seguinte.

Coluna de matriz

Matriz row-major com oito inteiros por linha:

```
passo entre elementos da mesma coluna=8.4=32 bytes
```

Stride prefetch pode aprender 32 bytes.

Pointer chasing

Se o próximo endereço está armazenado no dado atual e não segue padrão regular, prefetch simples possui pouca informação antecipada.

Custos:

- poluição da cache;
- consumo de largura de banda;
- ocupação de filas;
- energia;
- expulsão precoce de dado útil.

Um prefetch correto pode chegar tarde, sem esconder latência, ou cedo demais, sendo expulso antes do uso.

Cache responde onde manter cópias rápidas. Prefetch decide quando antecipar transferência. Memória virtual traduz e protege endereços. Branch prediction prevê fluxo de instruções.

13. Pipeline e hazards

13.1 Cinco estágios clássicos

Estágio	Função principal
IF	buscar instrução e calcular caminho sequencial do PC
ID	decodificar e ler registradores
EX	executar ULA, comparar ou calcular endereço
MEM	acessar memória de dados
WB	escrever resultado no banco de registradores

Uma instrução que não precisa de memória ainda atravessa conceitualmente o estágio MEM em um pipeline fixo. STORE e branch podem não produzir write-back útil; a convenção do diagrama define se esses estágios vazios são mostrados.

Registradores entre estágios mantêm dados e controles:

IF/ID
ID/EX
EX/MEM
MEM/WB

Sem esses registradores, resultados de instruções diferentes se misturariam no mesmo ciclo.

13.2 Sobreposição, fill e drain

Sem pipeline, uma instrução completa todas as etapas antes da próxima. Com pipeline, etapas de instruções distintas são executadas ao mesmo tempo.

Para k estágios e n instruções, no caso ideal:

$ciclos = k + n - 1$

Os $k-1$ ciclos extras representam enchimento e esvaziamento.

Exemplo com cinco estágios e quatro instruções:

Ciclo	I1	I2	I3	I4
1	IF			
2	ID	IF		
3	EX	ID	IF	
4	MEM	EX	ID	IF
5	WB	MEM	EX	ID
6		WB	MEM	EX
7			WB	MEM
8				WB

$ciclos = 5 + 4 - 1 = 8$

13.3 Latência, throughput, CPI e IPC

Latência é o tempo entre iniciar uma instrução e obter seu resultado. Throughput é a quantidade concluída por unidade de tempo.

```

latência aproximada=k.Tclock
throughput ideal após enchimento=1/Tclock
CPI=ciclos/instruções
IPC=instruções/ciclos

```

Em pipeline single-issue ideal e longo:

```

CPI aproxima 1
IPC aproxima 1
IPC aproxima 1/CPI

```

Pipeline melhora principalmente throughput. A latência de uma instrução pode permanecer semelhante ou até aumentar devido aos registradores de pipeline.

Tempo de CPU:

```

tempo=contagem de instruções.CPI.Tclock

```

Reduzir Tclock pode não compensar aumento de CPI ou contagem de instruções.

13.4 Período do clock e balanceamento

```

Tclock >= maior atraso de estágio + overhead do registrador

```

Exemplo:

```

atrasos dos estágios=250,150,200,300,180 ps
overhead do registrador=20 ps
Tclock mínimo=300+20=320 ps

```

Latência aproximada de cinco estágios:

```

5.320=1600 ps

```

Throughput ideal:

```

uma conclusão a cada 320 ps após enchimento

```

Os estágios de 150 ps e 180 ps possuem folga que não reduz o período, pois o estágio de 300 ps domina. Balancear estágios aproxima seus atrasos e melhora o uso do clock.

Speedup ideal em comparação com execução não sobreposta de k etapas iguais:

```

speedup=n.k/(k+n-1)

```

Para n muito grande, o limite aproxima k. Desbalanceamento, overhead e hazards reduzem esse valor.

13.5 Ciclos reais

```

ciclos reais=k+n-1
    + ciclos de stall
    + ciclos perdidos por flush
    + outras penalidades

```

Pipeline cheio pode concluir uma instrução por ciclo apenas quando:

- não há hazard estrutural;
- operandos chegam a tempo;
- fluxo de controle é conhecido ou previsto corretamente;
- cache e memória não introduzem espera;

- a largura de emissão é uma instrução por ciclo.

13.6 Hazard estrutural

Duas operações precisam do mesmo recurso no mesmo ciclo.

Exemplo:

```
uma única memória de uma porta
LOAD em MEM precisa ler dado
instrução seguinte em IF precisa buscar instrução
```

Soluções:

- stall;
- memória com mais portas;
- caches separadas de instrução e dados;
- duplicação ou escalonamento de recurso.

Hazard estrutural não é atraso de porta. É conflito de uso.

13.7 Dependências de dados

RAW, Read After Write:

```
instrução posterior lê valor que a anterior ainda produzirá
```

WAR, Write After Read:

```
escrita posterior não pode ocorrer antes da leitura anterior
```

WAW, Write After Write:

```
duas escritas no mesmo destino precisam preservar ordem
```

No pipeline in-order clássico de cinco estágios:

```
leituras ocorrem em ID
escritas ocorrem em WB
instruções avançam em ordem
```

Por isso WAR e WAW normalmente não aparecem. Elas surgem em arquiteturas com execução fora de ordem, latências variáveis ou múltiplas unidades.

13.8 Forwarding

Forwarding encaminha resultado já calculado diretamente ao consumidor, sem esperar sua escrita no banco.

Exemplo:

```
ADD R1,R2,R3
SUB R4,R1,R5
```

Quando SUB chega a EX, ADD pode ter o resultado em EX/MEM:

```
EX/MEM -> entrada da ULA em EX
```

Outro caminho:

```
MEM/WB -> entrada da ULA em EX
```

Condições conceituais:

```

produtor.RegWrite=1
produtor.destino diferente de registrador nulo, se houver
produtor.destino coincide com fonte do consumidor

```

Se EX/MEM e MEM/WB coincidem com a mesma fonte, o resultado mais novo geralmente é EX/MEM.

STORE ou branch sem destino não devem criar forwarding falso. O valor encaminhado precisa ser o dado correto, não apenas algum campo do registrador de pipeline.

13.9 Load-use

```

LOAD R1,[R2]
ADD R3,R1,R4

```

Cronologia:

```

LOAD em EX calcula endereço
LOAD em MEM recebe o dado no fim do estágio
ADD seguinte precisa de R1 no início de EX

```

O dado nasce tarde demais para o consumidor imediato. No pipeline clássico:

```
1 stall + forwarding
```

Uma instrução independente pode ocupar o intervalo:

```

LOAD R1,[R2]
ADD R6,R6,#1
ADD R3,R1,R4

```

Reordenação só é válida se preservar dados, flags, memória e fluxo.

13.10 Stall e bubble

Stall faz uma instrução correta esperar. Uma implementação típica de load-use:

```

congelar PC
manter IF/ID
inserir controles neutros em ID/EX

```

Os controles neutros formam uma bubble, uma posição sem efeito que percorre o pipeline.

Bubble não precisa corresponder a uma instrução NOP escrita no programa. É estado inválido interno.

13.11 Hazard de controle e flush

Branch cria incerteza sobre o próximo PC.

Estratégias:

```

esperar decisão -> stalls preventivos
prever e continuar -> execução especulativa

```

Se a previsão estiver errada, instruções jovens do caminho incorreto são invalidadas:

```
flush
```

Stall preserva a instrução correta. Flush descarta instruções que não devem executar.

O programador normalmente não escreve stall ou flush. A unidade de hazards e controle gera os sinais. Compilador pode reorganizar instruções ou inserir NOP em arquiteturas específicas.

A penalidade depende do estágio onde o branch é resolvido e de quantas instruções incorretas já entraram.

13.12 Efeito no CPI

delta CPI de branch=
frequência de branches.taxa de erro.penalidade

delta CPI de load-use=
frequência de load-use.stalls por ocorrência

Modelo combinado:

CPI=CPIideal
+ fbranch.erro.penalidade
+ floaduse.stalls
+ outras esperas médias

Diminuir a frequência de clock pode corrigir temporização elétrica, mas não elimina dependência lógica, miss de cache ou incerteza do próximo PC.

14. Predição de desvios

14.1 Direção e alvo

Um branch exige duas respostas:

```
direção -> tomado ou não tomado
alvo -> qual será o próximo endereço se tomado
```

Acertar direção e errar alvo ainda produz próximo PC incorreto.

```
tomado -> PC recebe alvo
não tomado -> PC segue caminho sequencial
```

Predição não altera a semântica do programa. Ela decide quais instruções buscar especulativamente.

14.2 Predição estática

Usa regra fixa, sem histórico de execução:

```
sempre não tomado
sempre tomado
branch para trás tomado, para frente não tomado
```

Branches para trás frequentemente fecham laços e tendem a ser tomados durante as iterações.

14.3 Preditor dinâmico de um bit

Guarda o último resultado:

```
estado N -> prevê não tomado
estado T -> prevê tomado
```

Após observar o branch, o estado vira o resultado real.

Em laço com padrão:

```
T,T,T,T,N
```

O preditor aprende tomado, erra na saída N e, na próxima entrada do laço, pode errar novamente ao prever N. Um único evento contrário muda a previsão imediatamente.

14.4 Contador saturante de dois bits

Estado	Nome	Previsão
00	fortemente não tomado	N
01	fracamente não tomado	N
10	fracamente tomado	T
11	fortemente tomado	T

```
resultado T -> incrementa
resultado N -> decrementa
satura em 00 e 11
```

O estado é consultado antes de ser atualizado.

Partindo de estado forte, dois resultados consecutivos contrários são necessários para mudar a direção prevista. Partindo de estado fraco, um basta. Essa histerese evita que uma única saída de laço reverta imediatamente o comportamento aprendido.

14.5 Histórico local e global

Histórico local registra comportamento de um branch específico. Pode capturar padrão periódico daquele branch.

Histórico global registra resultados recentes de vários branches:

GHR = Global History Register

Ele pode capturar correlação entre branches diferentes.

Gshare combina bits do PC com histórico global por XOR para indexar uma tabela de contadores. A combinação reduz alguns padrões de conflito, mas não elimina aliasing.

14.6 BHT, PHT, BTB e RAS

BHT, Branch History Table:

- indexada por bits do PC;
- pode guardar contador ou histórico, conforme a terminologia da arquitetura.

PHT, Pattern History Table:

- contém contadores associados a padrões de histórico.

BTB, Branch Target Buffer:

- reconhece branches vistos;
- fornece alvo previsto;
- não avalia a condição de CMP.

RAS, Return Address Stack:

CALL -> empilha endereço de retorno
RET -> usa e remove o topo

RAS trata retornos melhor que um único alvo por PC porque a mesma função pode ser chamada de vários lugares e recursivamente.

14.7 Aliasing, capacidade e aquecimento

Tabelas são finitas. Branches diferentes podem mapear para a mesma entrada:

aliasing

Se os comportamentos são opostos, eles interferem destrutivamente.

Warm-up é o período inicial em que tabelas e BTB ainda não possuem histórico útil. Troca de contexto pode preservar, contaminar ou limpar estado, conforme o projeto.

Limitações:

- capacidade finita;
- poucos bits do PC;
- histórico curto;
- padrões não estacionários;
- branches indiretos com múltiplos alvos.

14.8 Acurácia e custo

acurácia=predições corretas/branches
taxa de erro=1-acurácia

CPI aproximado=CPIideal
+ frequência de branch.taxa de erro.penalidade

Exemplo:

```
CPIideal=1  
branches=20%=0,20  
erro=10%=0,10  
penalidade=5 ciclos  
  
CPI=1+0,20.0,10.5=1,10
```

Maior acurácia tem mais valor quando branches são frequentes ou a penalidade é grande. Pipelines profundos costumam tornar misprediction mais caro.

15. FPU e ponto flutuante em hardware

15.1 Representação versus unidade funcional

IEEE 754 define formatos e comportamento numérico. FPU é o hardware que executa operações sobre esses formatos.

IEEE 754 -> representação e regras
FPU -> datapath, controle, arredondamento e flags

Fluxo geral:

desempacotar
-> classificar operandos
-> operar
-> normalizar
-> arredondar
-> empacotar

Antes da conta, a FPU identifica zero, subnormal, normal, infinito e NaN.

15.2 Soma e subtração

Etapas:

1. comparar expoentes;
2. deslocar à direita o significando de menor expoente;
3. somar ou subtrair significandos conforme sinais;
4. normalizar;
5. ajustar expoente;
6. arredondar;
7. tratar overflow, underflow e casos especiais.

Exemplo conceitual:

$1,100 \cdot 2^4$
 $1,010 \cdot 2^2$

Alinhar o segundo:

$1,100 \cdot 2^4$
 $0,01010 \cdot 2^4$

Somente depois os significandos são somados.

Na soma, expoentes não são somados. Eles são alinhados.

Carry e cancelamento

Se a soma dos significandos produz:

$10.xxxxx \cdot 2^e$

desloca-se à direita e aumenta-se expoente:

$1.0xxxxx \cdot 2^{(e+1)}$

Em subtração de valores próximos, zeros iniciais podem surgir:

$1,0000 \cdot 2^4 - 0,1111 \cdot 2^4$
 $=0,0001 \cdot 2^4$
 $=1,0000 \cdot 2^0$

Esse cancelamento exige deslocamento à esquerda e redução do expoente. Dígitos significativos podem ser perdidos por arredondamento anterior.

15.3 Guard, round e sticky

Durante alinhamento e normalização, bits descartados são resumidos por:

G = primeiro descartado
R = segundo descartado
S = OR de todos os posteriores

Nearest, ties-to-even:

incrementar se G=1 e
(R=1 ou S=1 ou LSB mantido=1)

Casos:

GRS=100 e LSB mantido 0 -> empate, mantém par
GRS=100 e LSB mantido 1 -> empate, incrementa para par
G=1 e R ou S=1 -> valor está além da metade, incrementa

15.4 Multiplicação

sinal=sA XOR sB
significando=significandoA.significandoB
expoente codificado=EA+EB-bias

Depois:

normalizar
arredondar
tratar especiais

O bias é subtraído uma vez porque já estava incluído nos dois expoentes armazenados.

Exemplo de expoentes em simples:

eA=3 -> EA=130
eB=2 -> EB=129
Eresultado=130+129-127=132
ereal=132-127=5

15.5 Divisão

sinal=sA XOR sB
significando=significandoA/significandoB
expoente codificado=EA-EB+bias

Divisão costuma ter maior latência ou implementação iterativa, embora unidades modernas possam ser profundamente otimizadas.

15.6 Casos especiais

Zero com sinal

Existem +0 e -0. Eles comparam como iguais em comparação numérica comum, mas o sinal pode influenciar:

1/+0=+infinito
1/-0=-infinito

Subnormal

Usa significando 0.f em vez de 1.f e expoente mínimo fixo. Permite aproximação gradual de zero, com menos bits significativos efetivos.

Infinito

Representa overflow e limites de operações válidas:

```
número finito não zero / 0 -> infinito com sinal, conforme regras
```

NaN

Representa resultado não numérico ou dado inválido.

Operações clássicas inválidas:

```
0/0
infinito-infinito
infinito.0
raiz de número negativo em reais
```

NaN não é igual a si mesmo em comparação comum:

```
NaN==NaN é falso
```

Comparações são unordered. Quiet NaN normalmente se propaga; signaling NaN pode levantar invalid.

15.7 Flags IEEE 754

```
invalid
divide-by-zero
overflow
underflow
inexact
```

Overflow indica magnitude grande demais para o formato após arredondamento. Underflow relaciona resultado minúsculo e perda relevante na região subnormal, conforme a definição de tininess adotada. Inexact indica que o resultado matemático não foi representado exatamente.

Flags de status podem permanecer ativas até serem limpas. Isso é diferente do bit sticky usado internamente no arredondamento.

15.8 Conversões e comparações

Conversões:

```
inteiro -> float
float -> inteiro
single -> double
double -> single
```

Single para double normalmente é exato porque double possui mais precisão e faixa. Double para single pode arredondar, gerar subnormal ou overflow.

Conversão numérica não é reinterpretação:

```
converter inteiro 1 para float -> padrão IEEE de 1,0
reinterpretar bits 0x00000001 como float -> subnormal minúsculo
```

Comparação precisa tratar NaN separadamente. Para números normais do mesmo sinal, expoente e significando determinam magnitude, mas uma comparação completa precisa considerar sinal, zeros e especiais.

15.9 Latência, throughput e pipeline

latência -> ciclos até o resultado
throughput -> frequência de aceitação ou saída
intervalo de iniciação -> ciclos entre novas operações

Uma unidade com latência quatro e intervalo de iniciação um pode iniciar uma operação independente por ciclo. Uma cadeia dependente precisa aguardar o resultado anterior.

FPU pode ter pipelines diferentes para soma, multiplicação e divisão. Registradores de ponto flutuante alimentam essas unidades.

SIMD aplica uma instrução a vários elementos. GPU contém muitas unidades paralelas e é eficiente para cargas com grande paralelismo. FPU, SIMD e GPU são conceitos relacionados, mas não equivalentes.

Dupla precisão usa mais bits de armazenamento e caminho de dados. Pode aumentar área, energia, largura de banda e custo, embora o impacto exato dependa do hardware.

16. Inglês instrumental e leitura técnica

16.1 Leitura instrumental

Leitura instrumental busca compreender estrutura, relações e informação necessária sem traduzir cada palavra.

skimming -> tema, organização e ideia geral
scanning -> termo, número, referente ou evidência específica

Texto técnico costuma apresentar:

contexto ou problema
lacuna
objetivo
método
resultado
conclusão

Reconhecer essa organização reduz dependência de vocabulário isolado.

16.2 Vocabulário consolidado

Palavra	Sentido técnico contextual
avenue/avenues	caminho, abordagem, possibilidade
encompass	abranger, incluir
faint	tênue, fraco, pouco intenso
fainter	mais tênue, menos brilhante/intenso
thus	portanto, assim
briefly	brevemente, de modo resumido
moreover	além disso
further	adicionalmente, além disso; também pode significar mais distante conforme o contexto

O sentido adequado preserva o domínio semântico. Em astronomia, fainter tende a se referir a brilho; em eletrônica, pode se referir a sinal menos intenso.

16.3 Ideia central e detalhe

Ideia central cobre o texto inteiro. Detalhe pode ser verdadeiro, mas explica apenas exemplo, causa, método ou consequência.

Uma formulação de ideia central normalmente contém:

assunto + ação ou relação principal + recorte

Pano de fundo introduz o problema, mas não é necessariamente o foco. Uma tecnologia anterior pode ser mencionada somente para motivar a proposta nova.

16.4 Conectivos por função

Função	Conectivos
adição	moreover, furthermore, further, in addition, also
contraste	however, nevertheless, although, whereas, while
causa	because, since, due to, owing to
consequência	therefore, thus, hence, consequently
exemplo	for example, for instance, such as
resumo	in short, in summary, overall
sequência	first, then, subsequently, finally
condição	if, unless, provided that
concessão	although, even though, despite

Conectivo define relação lógica, não apenas tradução.

further e moreover -> adição
 thus e therefore -> consequência
 however -> contraste
 in short -> resumo

Briefly é frequentemente advérbio de modo:

the paper briefly discusses recent advances
 objetivo da ação -> discutir avanços recentes
 modo -> brevemente

Briefly não equivale automaticamente a in short.

16.5 Paráfrase

Paráfrase correta preserva:

- agente;
- ação;
- objeto;
- relação lógica;
- intensidade;
- modalidade;
- restrições.

Equivalências contextuais:

properties -> characteristics
 include -> encompass
 fainter -> less bright
 further -> moreover

Repetir palavras não garante equivalência. Mudanças pequenas podem alterar sentido:

may improve -> pode melhorar
 improves -> melhora como fato
 must improve -> precisa melhorar

16.6 Comparativos e magnitude

Estruturas:

faster than
 more efficient than
 less expensive than
 as accurate as
 millions of times fainter than

Times indica fator de magnitude:

twice as large -> duas vezes tão grande
 three times faster -> expressão contextual de fator três

Comparação precisa preservar o eixo: velocidade, custo, brilho, potência, precisão ou outra grandeza.

16.7 Objetivo de texto acadêmico

Marcadores:

this paper proposes...
 the aim of this study is...
 we investigate...
 this work presents...
 the authors evaluate...

Separação gramatical:

verbo principal -> ação/objetivo
 objeto -> tema estudado
 advérbio -> modo ou intensidade

Em:

this work briefly presents a low-power architecture

 objetivo -> apresentar uma arquitetura de baixa potência
 briefly -> modo da apresentação

16.8 Modalidade, intensidade e tom

Modalizadores:

may, might, could -> possibilidade
 likely, probably -> probabilidade
 should -> expectativa/recomendação
 must -> obrigação ou conclusão forte

Intensificadores:

significantly
 remarkably
 substantially
 highly
 slightly

Tom pode ser:

neutro
positivo
crítico
cauteloso
otimista
cético
humorístico

Texto científico frequentemente usa modalidade para limitar afirmações. Retirar may ou likely transforma hipótese em certeza.

16.9 Referência pronominal

Pronome	Referente típico
it	coisa, processo ou ideia singular
they	plural
this/that	item ou proposição próxima
who	pessoa
which	coisa ou ideia
that	pessoa/coisa em oração restritiva
whose	posse
where	lugar ou contexto espacial

Antecedente precisa concordar em número e sentido.

Oração restritiva identifica qual elemento:

devices that operate at low voltage...

Oração explicativa, normalmente entre vírgulas, acrescenta informação:

the device, which was fabricated in 28 nm, consumes...

Remover uma restritiva pode mudar a referência. Remover uma explicativa preserva a identificação principal.

16.10 Voz passiva e relações técnicas

Voz passiva enfatiza processo ou resultado:

the circuit was fabricated in CMOS
the samples were measured at room temperature

O agente pode ser omitido. Isso não significa ausência de ação; muda o foco informacional.

Participles e nominalizações são comuns:

measured values
proposed method
power reduction
signal processing

Compostos técnicos devem ser lidos como unidade:

low-power design
high-speed interface
clock-gating technique

16.11 Figuras de linguagem e humor

Metáfora transfere sentido:

bottleneck -> gargalo de desempenho
pipeline bubble -> posição sem trabalho útil

Metonímia usa entidade relacionada:

the paper argues... -> os autores argumentam

Antítese aproxima ideias opostas. Pleonasma repete sentido. Humor técnico pode depender de estereótipos profissionais, contraste ou duplo sentido.

Interpretação deve separar significado literal da função comunicativa.

16.12 Erros semânticos recorrentes

Uma afirmação pode ser inadequada por:

copiar palavras e mudar o foco
usar detalhe verdadeiro como síntese geral
acrescentar informação não sustentada
trocar causa por consequência
trocar contraste por adição
generalizar além do texto
restringir mais do que o texto
transformar possibilidade em certeza
confundir modo com objetivo

Essas categorias descrevem relações semânticas gerais e ajudam a explicar por que duas frases lexicalmente próximas não são equivalentes.

17. Referência consolidada e conexões

17.1 Quantidade, largura e capacidade

n bits $\rightarrow 2^n$ padrões
 tabela com n variáveis $\rightarrow 2^n$ linhas
 decoder n para 2^n
 MUX com 2^n entradas $\rightarrow n$ seletores
 n bits de endereço $\rightarrow 2^n$ posições
 capacidade em bits=posições.largura
 LUT de k entradas $\rightarrow 2^k$ posições
 N estados binários $\rightarrow \lceil \log_2 N \rceil$ flip-flops
 N estados one-hot $\rightarrow N$ flip-flops
 ring com n flip-flops $\rightarrow n$ estados válidos padrão
 Johnson com n flip-flops $\rightarrow 2n$ estados válidos padrão

17.2 Faixas e aritmética

unsigned n bits=0 até 2^n-1
 C2 n bits= $-2^{(n-1)}$ até $2^{(n-1)}-1$
 $A-B=A+B'+1$
 $C=1$ após subtração ARM-like $\rightarrow$ sem borrow
 V soma=carry de entrada do MSB XOR carry de saída

 valor fixo=inteiro bruto/ 2^F
 resolução fixa= 2^{-F}

17.3 Booleana e blocos

$(A+B)' = A'B'$
 $(AB)' = A' + B'$
 $XOR = A'B + AB'$
 $XNOR = AB + A'B'$
 MUX 2:1: $Y = S'I_0 + SI_1$

 half-adder:
 $S = A \oplus B$
 $C = A.B$

 full-adder:
 $S = A \oplus B \oplus C_{in}$
 $C_{out} = A.B + A.C_{in} + B.C_{in}$

17.4 Temporização

Com skew definido como chegada no destino menos chegada na origem:

$T_{clock} >= t_{CQmax} + t_{combmax} + t_{setup-skew}$
 $t_{CQmin} + t_{combmin} >= t_{hold} + skew$

 $F_{max} = 1/T_{clock_min}$
 $F_{max}(MHz) = 1000/T(ns)$

 ripple:
 atraso da soma do MSB = $(n-1)t_{carry} + t_{soma}$

17.5 Energia e potência

$V = R \cdot I$
 $P = V \cdot I$
 $E = P \cdot t$
 $Q = C \cdot V$

 $P_{din} \text{ aproximada} = \alpha \cdot C \cdot VDD^2 \cdot f$
 $P_{est} \text{ aproximada} = VDD \cdot I_{leak}$
 $E_{cap} = 1/2 \cdot C \cdot V^2$
 $E_{fonte} \text{ no carregamento } 0 \text{ para } V = C \cdot V^2$

17.6 Cache, memória virtual e prefetch

$miss \text{ rate} = 1 - hit \text{ rate}$
 $conjuntos = capacidade / (bloco \cdot vias)$
 $offset = \log_2(\text{bytes por bloco})$
 $índice = \log_2(conjuntos)$
 $tag = \text{bits restantes}$
 $AMAT = hit \text{ time} + miss \text{ rate} \cdot miss \text{ penalty}$

 $endereço \text{ virtual} = VPN + offset$
 $endereço \text{ físico} = PFN + o \text{ mesmo offset}$

 $accuracy \text{ de prefetch} = \text{úteis} / \text{realizados}$
 $coverage = \text{misses evitados} / \text{misses originais}$

17.7 Pipeline e branch

$ciclos \text{ ideais} = k + n - 1$
 $ciclos \text{ reais} = ideal + stalls + flushes + outras \text{ esperas}$
 $CPI = ciclos / instruções$
 $IPC = instruções / ciclos$
 $tempo = IC \cdot CPI \cdot T_{clock}$

 $\Delta CPI \text{ de branch} = f_{branch} \cdot taxa \text{ de erro} \cdot penalidade$

17.8 Conversões de unidade

$1 \text{ byte} = 8 \text{ bits}$
 $word \text{ de } 32 \text{ bits} = 4 \text{ bytes}$

 $1 \text{ ms} = 10^{-3} \text{ s}$
 $1 \text{ microsegundo} = 10^{-6} \text{ s}$
 $1 \text{ ns} = 10^{-9} \text{ s}$
 $1 \text{ ps} = 10^{-12} \text{ s}$

 $1 \text{ kHz} = 10^3 \text{ Hz}$
 $1 \text{ MHz} = 10^6 \text{ Hz}$
 $1 \text{ GHz} = 10^9 \text{ Hz}$

 $1000 \text{ ps} = 1 \text{ ns}$
 $T(\text{ns}) = 1000 / f(\text{MHz})$
 $f(\text{MHz}) = 1000 / T(\text{ns})$

17.9 Relações entre as camadas

tabela-verdade

- > mintermos/maxtermos
- > Karnaugh
- > expressão
- > portas
- > rede CMOS
- > atraso e energia

latch

- > flip-flop
- > registrador
- > FSM e contador
- > datapath e pipeline

algoritmo

- > ISA/assembly
- > sinais de controle
- > datapath
- > pipeline
- > cache e memória física

endereço virtual

- > TLB/MMU
- > endereço físico
- > cache
- > DRAM
- > armazenamento em page fault

CMP

- > ULA calcula subtração
- > flags Z,N,C,V
- > branch escolhe próximo PC
- > preditor tenta antecipar direção/alvo
- > flush corrige especulação errada

LOAD

- > assembly calcula EA
- > TLB traduz
- > cache procura bloco
- > memória pode fornecer dado
- > pipeline pode sofrer load-use
- > forwarding entrega valor ao consumidor

ponto fixo

- > somador/shifter/ULA

ponto flutuante

- > formato IEEE 754
- > FPU alinha, opera, normaliza e arredonda
- > pipeline da FPU determina latência e throughput

Essas relações mostram que o mesmo evento pode ser descrito em várias camadas sem que os conceitos sejam equivalentes. Um LDR é uma instrução; hit ou miss é comportamento da cache; TLB hit ou miss é comportamento da tradução; load-use é uma dependência de pipeline; page fault é intervenção do sistema operacional.